

lences, and special I/O condition keywords. The Fortran equivalence imposes special conditions on the memory allocator that are not found in any other language. We see that memory allocation is radically different in the three languages.

5. Algol has no I/O conventions. Fortran and Basic do support I/O conventions, and the two conventions are quite different. Clearly, a common IL must effectively support two different I/O systems.

These examples of language differences should be sufficient to make our central point: an IL that must support several languages must contain the union set of all those language features, which will make the IL considerably more complicated than it would be for any one of the languages alone. Many of the features must be identified as peculiar to one or another language because of semantic differences. Some language features, particularly the declaration, I/O, and block structuring conventions, require such specialized treatment that there might as well be a different IL for each language.

Some problems arise in the design of back ends with multiple machines and one IL, but they are less severe than the multiple language problem. If the IL is consistently designed, each of its operations can somehow be implemented on a target machine of reasonable capability. Since the IL can in principle be expressed as an abstract syntax tree, there is no reason that efficient code could not be generated from a given IL for several different machines.

Nevertheless, certain optimization problems may occur. For example, array indexing may be supported very efficiently by special mechanisms on one computer but not another. If indexing is expressed by a sequence of primitive triples, with dimension multiplication and addition, etc., it would be difficult to recognize as an operation that could be efficiently performed on one machine. On the other hand, if every such "high-level" operation has a special notation in an IL, then each of the compiler back ends must effectively take them into account, and their complexity will increase. The Cobol language in particular requires a large number of special operations that are directly supported in hardware on some machines and not on others.

Now suppose that an IL is designed with, say, three languages in mind. When a fourth language is to be added to the IL at some later date, the existing IL will likely have to be augmented with some new features. Then each of the back ends must be extended to support the new IL features. This incremental effort will likely be less than that needed for a new compiler, but will not be insignificant either.

The design and implementation of an IL system is a cost in addition to those incurred in a more conventional compiler design. High-level design effort is often more costly than coding, since competent software designers demand higher salaries than coders.

On the surface, an IL is best suited to a multipass compiler; however, it can also be used in a one-pass or interactive compiler. In the latter case, the IL is

simply a software data structure through which all the information must flow in passing from source to object code.

We conclude that an IL may be practical for a few closely related languages and several different target machines. When practical, its use promotes transportability and reduced compiler writing costs, at the loss of some compile-time and run-time efficiency.

## 10.4. Machine architectures

There are several advantages to a compiler that generates target machine code directly, rather than through an IL. Such a compiler will usually be more efficient and generate more efficient object code. Of course, if a compiler is funded by a computer manufacturer, the manufacturer obviously will be interested only in its machines as target machines. A compiler that can generate code for both its own and its competitor's machines is not likely to be greeted with enthusiasm.

For whatever reason, many compilers are expected to generate instructions tailored to a specific target machine, and that machine's architecture is often not under control of the compiler writer. Of course, a back end of an IL is tailored to a specific machine. We therefore direct our attention to some issues in machine architecture and their relation to language translators.

A compiler that generates specific target machine code seldom generates only that; usually, a set of loader tables must be generated. These tables contain sequences of target machine instructions, but also carry information needed to link separately compiled program segments together, or to relocate the machine instructions after compilation. Relocation data is essential for multiprogrammed target machines that do not handle relocation automatically in its hardware. The IBM 360 is an example of a machine that requires attention to relocation in its translators; the CDC 6400 and HP 3000 are examples of machines that do not.

Certain features of a target machine system are of special interest to the designer of the code generators of a compiler:

1. *The loader table structure.* Only by generating loader tables can the resulting programs enjoy the advantages of relocation and partial compilation.
2. *The register and operating system conventions.* The IBM 360 requires the observance of certain conventions of every program to be executed in its system. Failure to observe these conventions will result in bounds violations or inability of the operating system to diagnose program problems.
3. *Data and program addressing.* How are data and program accessed in the system? Sometimes the addressing convention is very simple, e.g., the

CDC 6400, in which every memory reference instruction carries a full 18-bit address field. In other machines, the addressing convention may be rather complicated and may require special attention. As we have seen for the AOC machine, effective addressing for an Algol-like language requires a set of base registers (the display), instructions that carry an offset from a selected base register, and optional indexing on top of the base register contents and offset. Heap management and access requires double indirection through address labels. Uniform address labels whose nature can be inferred at run-time are very desirable for procedure calls. When such features are missing in the machine hardware, their equivalent must be organized through in-line instruction sequences or procedure calls, often with a sizable loss in performance. Fortran requires much less addressing structure than Algol and can be efficiently supported on most machine architectures. Of course, Fortran does not support recursive procedure calls, nested blocks, or dynamically alterable arrays.

4. *Completeness and uniformity of the instruction set.* Suppose that the system supports four data types. Are all four arithmetic operations supported on all four data types? Are they supported in a uniform manner, or are there special exceptions? Every exception to a general rule requires special attention in some part of the compiler. An operation that only applies to a subset of the class of data types will often have to be supported through a run-time procedure or in-line code sequence for the nonsupported types.
5. *Special instructions.* Often, the instruction set contains some instructions that are technically unnecessary for completeness but that can be exploited for the sake of coding efficiency. An increment-memory instruction is an example.

The designer of a code generator must be intimately familiar with the machine's instruction set. If he is not familiar with the machine, he should spend some time writing assembly language programs and studying the instruction coding conventions and the system conventions. Instructions sometimes have undesirable side-effects or may not perform exactly as expected. Such seemingly minor considerations must be carefully examined in the light of their use in the compiler.

We cannot possibly review all the known machine architectures and their relation to compiler design; there are simply too many. Instead, we shall review three machine architectures in some detail as case studies. These are the Hewlett-Packard 3000, the Control Data 6000, and the International Business Machines 360 systems. Of these, the HP-3000 will receive the most attention; it is a stack machine that incorporates many of the AOC machine notions and its architecture and loader supports partial compilation and program relocation.

The Control Data and IBM machines are examples of multiregister machines. Neither machine supports a push-down stack directly in their instruction set definitions as does the HP-3000, but both can support the AOC machine features without much loss in performance.

Each of the three machines has certain deficiencies that create difficulties for the designer of a code generator. We shall examine both the strengths and weaknesses of each system only from the point of view of overall compiler effectiveness. We do not claim that this is the only or the best basis for the comparison of computer systems. The evaluation of computer system performance must take many other factors into account and weigh them against the initial and maintenance costs of the system.

## 10.5. The Hewlett-Packard HP-3000

The Hewlett-Packard 3000 series III computer contains several AOC machine features in its instruction set. It is a general-purpose data processing system, specifically for time-shared multiprogramming applications. It is organized around a data stack and a separate instruction memory. Its basic word size is 16 bits, and it is therefore considered a “minicomputer”. However, its instruction set contains many powerful arithmetic and stack manipulation instructions, and its operating system provides sophisticated file management and other services.

Since we are principally interested only in those aspects of the HP-3000 that are of direct concern to a compiler designer, we shall limit our discussion to those features. Further information is given in the HP-3000 Computer System Reference Manual.

### 10.5.1. Data formats

A variety of numeric data types are supported in the instruction set, including complete arithmetic and 16-bit logical operations on 16-bit words, fixed-point and floating-point arithmetic on 32-bit double words, and arithmetic for a 64-bit floating-point number. Special instructions also provide certain data conversions from one number form to another. Packed decimal arithmetic is also supported. Because of the stack architecture, essentially all the operations are performed on the top elements of the stack. For example, a real multiply causes the floating-point number in locations  $\{s-1, s\}$  to be multiplied by a floating-point number in  $\{s-3, s-2\}$ . The two-word result replaces these four words on the stack.

The data types are indistinguishable at run-time, hence it is the compiler's (or user's) responsibility to keep track of the various data types supported by the source language and to emit the necessary conversion instructions.

Some half-word, or “byte” operations are also provided. Bytes may be

packed two per word, and arranged in arrays. We shall not discuss byte and string operations or the special byte addressing modes available.

All the data types (except byte) must be stored on an even-byte boundary (any 16-bit word boundary), owing to the addressing conventions of the machine. The word “must” should be qualified, inasmuch as it is possible to move an odd-byte variable from memory to an even-byte stack top location for an operation, then back again; but such an operation adds considerably to the operation time.

### 10.5.2. Memory and Register Organization

The machine’s memory organization (figure 10.2) is similar to that of the AOC machine. A program consists of one or more code segments; a code segment contains one or more procedures. Only one code segment is active at any one time, although more inactive code segments belonging to the program may also happen to reside in memory. Generally, inactive code segments are kept in a *program file* in disk and are fetched as needed through procedure calls.

A code segment is delimited by two registers, PB and PL (figure 10.2.) The current instruction location is marked by register P. The system enforces the restriction  $PB \leq P \leq PL$  and causes a program abort if this restriction is ever violated.

One data segment is normally provided (figure 10.2), and is defined by the following registers.

- DL: Data limit. Defines the least memory location.
- DB: Data base. Defines the “0” memory location. Negative locations are toward DL and positive locations are toward Z. DB corresponds to location 0 in the AOC machine, or D(0) in the AOC display.
- Z: Stack limit. Defines the largest memory location. This limit is automatically adjusted by the operating system as needed.
- S: Logical top-of-stack. Defines the topmost stack word currently in use. Many of the instructions either directly or indirectly affect S. However, an attempted access of data memory greater than S or less than DL results in a program abort.
- SM: Top-of-stack in memory. Of no concern to a user; the system carries the top four stack elements in fast electronic registers that track the memory contents. This register is used in that system. SM is not accessible through the instruction set.
- Q: Stack marker. On a procedure call, the Q-register is set to point to the new stack marker top. Register Q corresponds to D(b) – 1 in the AOC machine. There is no display register set.

A 16-bit index register is also provided for array indexing.

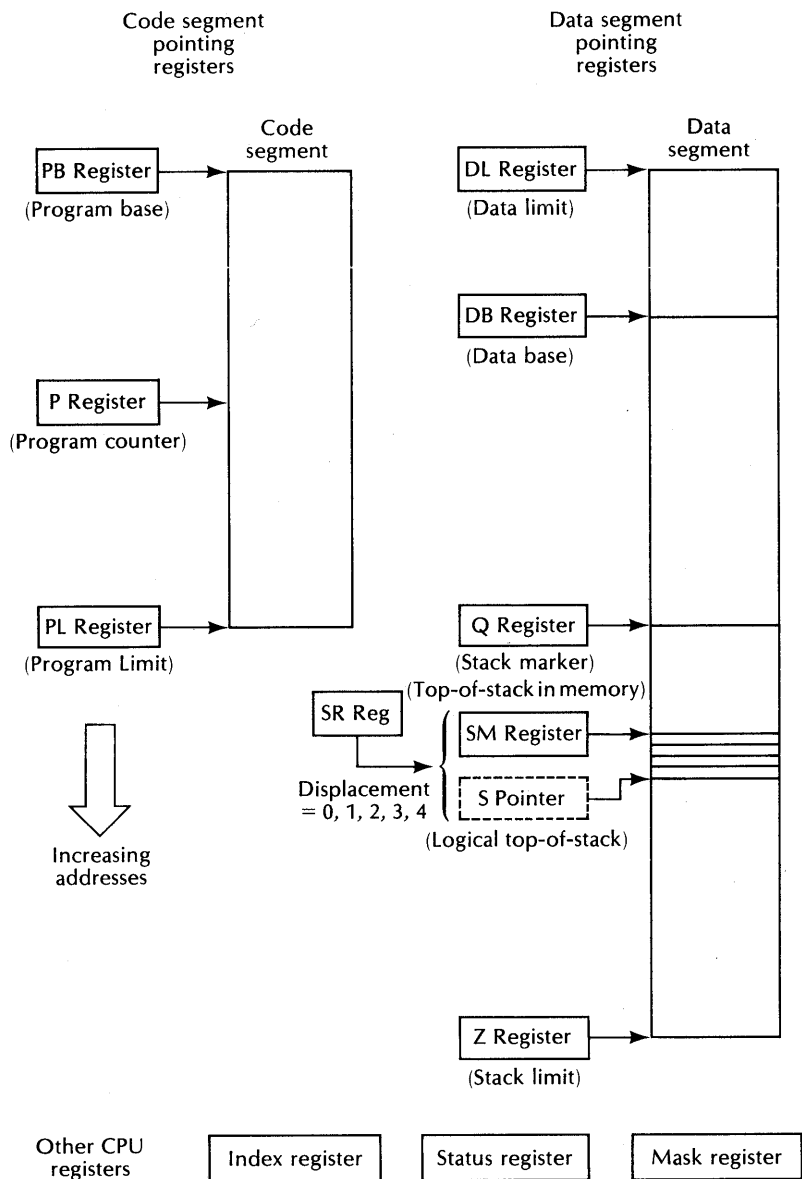


Figure 10.2. HP3000 memory organization and segment-pointing registers.

| Address mode  | Instruction bits |   |                        |                        |                       |    |    |    |    |    |
|---------------|------------------|---|------------------------|------------------------|-----------------------|----|----|----|----|----|
|               | 6                | 7 | 8                      | 9                      | 10                    | 11 | 12 | 13 | 14 | 15 |
| P + Relative  | 0                | 0 | ← Displacement 0:255 → |                        |                       |    |    |    |    |    |
| P - Relative  | 0                | 1 | ← Displacement 0:255 → |                        |                       |    |    |    |    |    |
| DB + Relative | 1                | 0 | ← Displacement 0:255 → |                        |                       |    |    |    |    |    |
| Q + Relative  | 1                | 1 | 0                      | ← Displacement 0:127 → |                       |    |    |    |    |    |
| Q - Relative  | 1                | 1 | 1                      | 0                      | ← Displacement 0:63 → |    |    |    |    |    |
| S - Relative  | 1                | 1 | 1                      | 1                      | ← Displacement 0:63 → |    |    |    |    |    |

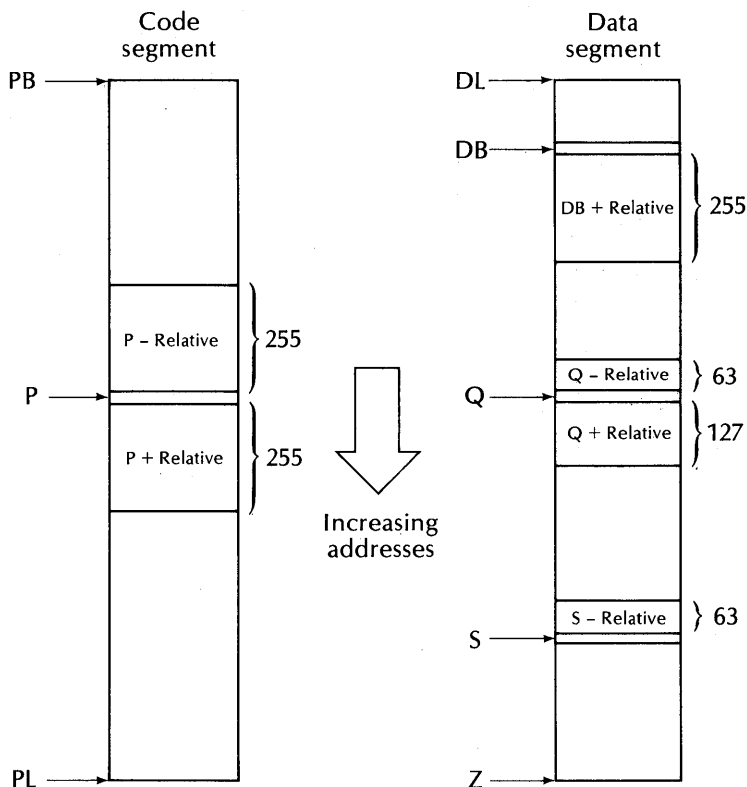


Figure 10.3. HP3000 memory direct addressing modes.

### 10.5.3. Memory reference instruction format

Twenty-one memory reference instructions are provided in the machine. These have the general form shown in figure 10.3. Bits 6 through 15 of the 16-bit word are used for a direct memory reference, through one of the registers P, DB, Q, or S. Bits 8 through 15 yield a word displacement field for P and DB relative addresses. Bits 9 through 15 yield the displacement for Q+ relative, and bits 10 through 15 yield the displacement for Q- and S-

relative addresses. The directly addressable memory fields are shown in the lower part of figure 10.3.

The DB+ relative field is called *primary DB*. It is used for global or outermost block variables. The primary DB addressing space is only 256 words, hence a large program with many global variables requires an additional capability—indirection or indexing.

The Q− relative field is principally used for formal parameter references. Its maximum size is 63 words, 4 of which are preempted by the stack marker.

The Q+ relative field is used for local variables, at the innermost block level. Its maximum size is 127 words.

The S− relative addressing can be used to access temporary cells whose location is only known relative to the stack top. For example, the RAV instruction in the AOC machine calls for a  $C(S-1)$  access.

The P+ and P− relative addressing modes are used in branch and load constant instructions. Access of locations greater than 255 words distant from the instruction requires indirection or indexing.

The maximum code segment size is imposed by the operating system, and also by certain bit fields defined in the system; the upper size limit is 16,384 words. The maximum data segment size (Z−DL) is approximately 32,000 words.

Several code and data segments may reside in physical memory simultaneously. All address references are relative to one of the registers shown in figure 10.3, hence their absolute location in memory is only of concern to the operating system memory manager.

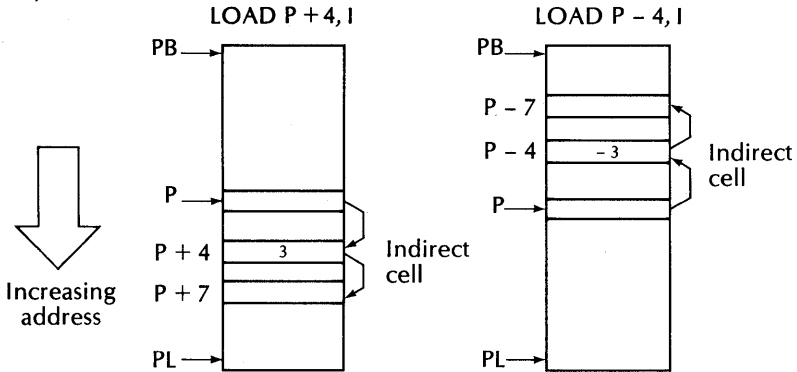
The code segment words are fixed before execution and cannot be changed by a user program. Because of this property, a given code segment can be shared by more than one user in a multiprogramming environment, and a code segment need never be written to disk, once it is completed by the compiler, linker, and loader. (A copy already exists on disk.) A data segment, however, must be written into a scratch area on disk upon transferring control of the system to another user.

#### 10.5.4. Indirection and Indexing

Because the range of direct addressing is so small, indirection and indexing are provided for the memory access (MA) instructions. In general, bit 4 of the instruction indicates indexing, and bit 5 indicates indirection. (There are a few exceptions.) Indirection is to only one level; the direct cell may only point to a data cell, not to another indirect cell. Indirection is illustrated in figure 10.4, in code and data space. For example,  $\text{LOAD } Q+4, I$  causes a fetch of the word in  $Q+4$  which happens to be 7. The 7 is DB relative, hence the accessed data word is in  $\text{DB}+7$ . An indirect cell is a 16-bit word, interpreted as a two's complement word count relative to DB. Hence, the negative locations toward DL can be accessed indirectly, as can all the stack area. The



CODE, Indirect



DATA, Indirect

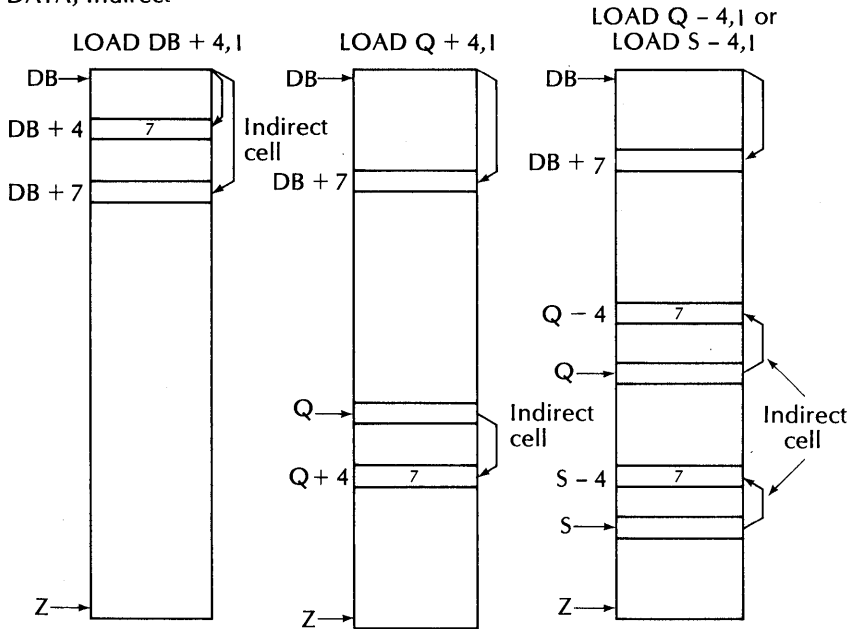


Figure 10.4. Examples of indirect addressing for HP3000.

maximum addressable indirect space is therefore  $DB \pm 32,767$ . However, the operating system limits the DL to Z size to approximately 32,000 words.

Indexing through the index register X provides an additional level of memory access. Indexing always follows an indirection, if any, and is illustrated in figure 10.5. For example, the instruction `LOAD Q+4,I,X` first accesses word `Q+4`, which contains 3; with  $C(X)=5$ , the data word at `DB+10` (octal) is accessed.

We now see how a data space in excess of 256 words (or whatever) can be allocated and accessed. If a global data area of (say) 500 words is needed for an array, it may be located somewhere above DB (but below the lowest stack marker) and accessed through an indirect word between DB and DB+255. Similarly, local arrays may be allocated above Q somewhere and accessed through an indirect word between Q+1 and Q+127.

Long branches can also be implemented by a short indirect branch through a word in the program memory. An indirection through a word at P+4 that contains “3” refers to location P+7 (see figure 10.4). That is, the indirect cell is relative to itself, just as the direct instruction displacement is relative to itself.

Self-relative instructions and indirections mean that all of the code of a procedure may be fixed by the compiler and need not be changed when the procedure code is positioned arbitrarily within a code segment. (This statement is not quite correct, as we shall see; e.g., procedure calls cannot be fixed by the compiler.)

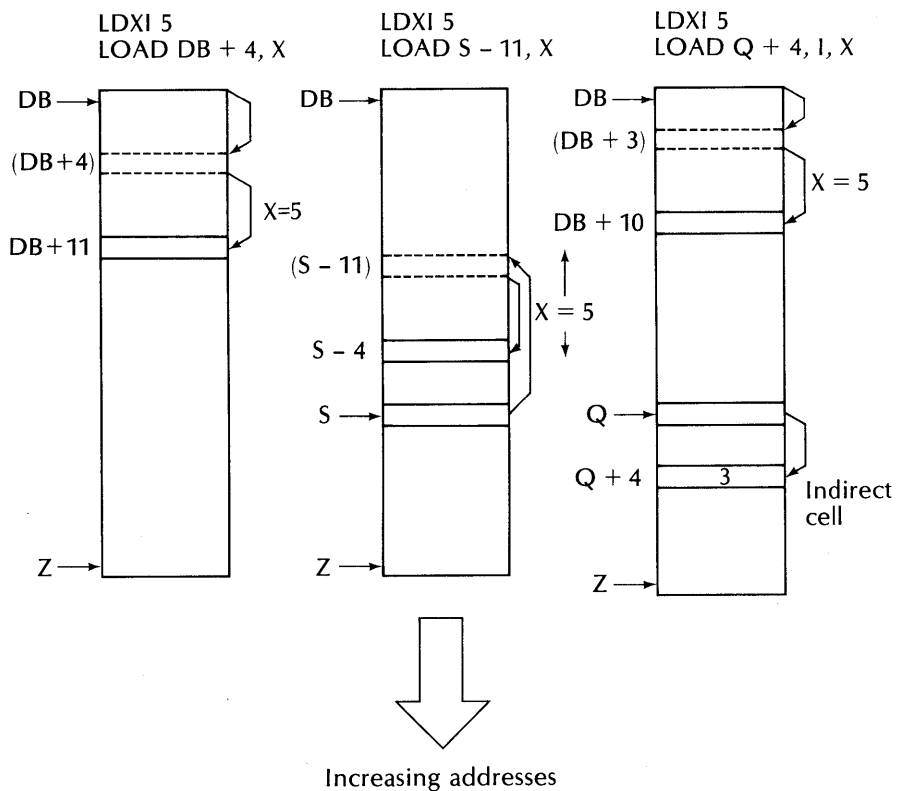


Figure 10.5. Examples of indirect and indexed addressing for HP3000.

### 10.5.5. Stack Configuration During Program Execution

Figure 10.6 illustrates a typical machine stack configuration during execution. Primary and secondary DB (*global data area*) are organized and initialized prior to execution. An initial stack marker is also placed by the system, and an EXIT through that marker returns the program to control of the operating system.

On a procedure call A, actual parameters are loaded on the stack prior to the call. The call places another stack marker; this marker contains a reference to the previous marker. Q points to the new marker. Local variables may then be built up on the stack. If A calls procedure B, this process is repeated, etc. Three procedures (A, B, and C) are currently active in figure 10.6, and control currently resides in procedure C, called by B. Of course, temporary storage is available between the local variables of one procedure and the actual parameters of the next call.

This machine automatically generates a stack marker, links it to the previous marker, and sets Q, upon executing a PCAL (procedure call) instruction. The PCAL also performs certain other useful services, as we shall see. Upon executing an EXIT instruction, Q is reset to the previous stack marker, and the stack is cut back.

### 10.5.6. Stack Marker, Procedure Calls and Exits

The stack marker is shown in figure 10.7. It carries the index register (X) contents at the moment of call, a PB-relative return address, some status information, a code segment number (more about that next), and a reference to the previous marker ("Delta Q").

By writing C(X) in the stack marker and subsequently restoring it upon a return, C(X) is effectively unchanged by a procedure call.

Among the status information is "O", an arithmetic overflow bit, "C", an arithmetic carry bit, and "CC", a condition code. The condition code CC may carry one of three values 0, 1, 2 (3 is not used). Code CC is set by a variety of instructions, and is tested in one form of conditional branch.

We now show how the HP-3000 system handles procedure calls and exits. Recall that all the code for some procedure resides in a code segment, that a program may consist of several code segments, and that several procedures may be placed in one segment.

The system permits procedures to be moved around from one code segment to another without recompiling the procedures. This is an important performance consideration, as we shall see, and is well worth the complicated mechanism that makes it possible.

Every procedure call is made through a *segment transfer table*, or STT (figure 10.8). One STT resides at the end of each code segment. It is a directory to the procedures. There is one STT entry, a *procedure label*, for

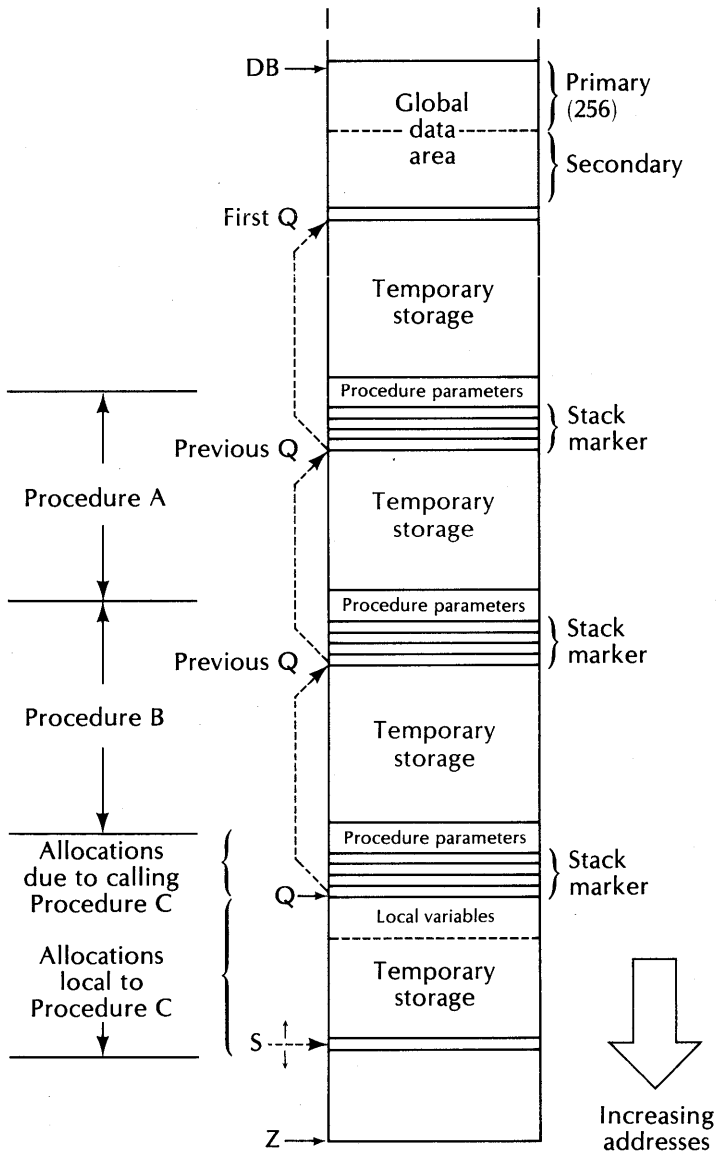


Figure 10.6. Typical stack configuration.

each procedure called in the segment. Different calls to the same procedure share the same STT entry. The STT entries are indexed backwards in memory, starting at the last word of their host code segment.

The PCAL instruction carries an index  $N$  ( $1 < N < 255$ ) into the STT table. STT(0) carries the largest STT number, not a procedure label.

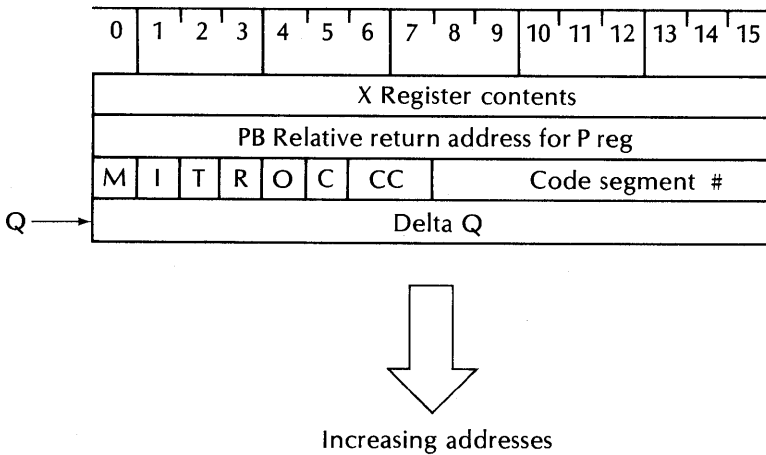


Figure 10.7. Stack marker format.

A procedure label has two forms, depending on whether its procedure is in its own code segment or not. Bit 0 specifies the form (see figure 10.9 for the general format.) Bit 1 (*uncallable* bit) is used for protection of operating systems procedures against nonprivileged calls, thus a 14-bit PB-relative address is available for a procedure call. If the procedure being called is in this code segment (*local program*), then the label carries its PB-relative entry location; a transfer to that location, along with the usual stack marker creation, etc., is made. In figure 10.8, the PCAL(4) in code segment 23 (path 5) is to a local procedure, whose label is in STT index 4. Thus control passes to some code location in segment 23 (see path 6).

If the procedure is in some other code segment (*external procedure*), there are two possibilities, shown by the PCAL(5) in code segment 23, to a procedure in code segment 22. Segment 22 may or may not currently reside in memory. (It may have been overwritten by the memory manager in order to provide space for other code or data memory segments.) This information is in a master table called the *code segment table* (CST). The CST is shared by all users of the system and is strictly managed by the operating system. It is accessed through a CST pointer in location 0 of absolute memory.

The CST entries are double words; their structure is shown in figure 10.9. Bit A is set if the code segment is absent. When it is absent, the memory manager must be called into service to allocate enough contiguous space for the segment (the length field is part of the CST entry). When space is available, an absolute disk address in the CST entry is accessed to load the new code segment. The old segment is marked "inactive," hence its space becomes available. When the new segment has been fully written to memory, the system transfers control to the procedure entry point, and adjusts the



**Code segment table** Doubleword

|         |   |   |   |        |   |   |   |   |   |    |    |    |    |    |    |
|---------|---|---|---|--------|---|---|---|---|---|----|----|----|----|----|----|
| 0       | 1 | 2 | 3 | 4      | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
| A       | M | T | R | Length |   |   |   |   |   |    |    |    |    |    |    |
| Address |   |   |   |        |   |   |   |   |   |    |    |    |    |    |    |

**A** Absence bit (= 1 if segment is absent)

**M** Mode bit (= 1 if privileged mode)

**T** Trace bit (= 1 to call Trace routine)

**R** Reference bit (for statistical use by operating system, set to 1 when accessed)

**Length** This value times 4 (max = 16,380)

**Address** Absolute memory address (for PB) or absolute disc address if absent

**Segment Transfer Table** Words

Local Program Label

|   |   |         |   |   |   |   |   |   |   |    |    |    |    |    |    |
|---|---|---------|---|---|---|---|---|---|---|----|----|----|----|----|----|
| 0 | 1 | 2       | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
| 0 | U | Address |   |   |   |   |   |   |   |    |    |    |    |    |    |

**U** Uncallable bit

**Address** PB relative, + only

External program label

|   |       |   |   |   |   |   |   |   |       |    |    |    |    |    |    |
|---|-------|---|---|---|---|---|---|---|-------|----|----|----|----|----|----|
| 0 | 1     | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9     | 10 | 11 | 12 | 13 | 14 | 15 |
| 1 | STT # |   |   |   |   |   |   |   | SEG # |    |    |    |    |    |    |

**STT #** STT entry number in target segment, maximum = 127

**SEG #** Target segment

**Status Word**

|   |   |   |   |   |   |    |         |   |   |    |    |    |    |    |    |
|---|---|---|---|---|---|----|---------|---|---|----|----|----|----|----|----|
| 0 | 1 | 2 | 3 | 4 | 5 | 6  | 7       | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
| M | I | T | R | O | C | CC | Segment |   |   |    |    |    |    |    |    |

**M** Mode bit (= 1 for privileged mode)

**I** Interrupt enable (1)/disable (0), external

**T** Traps enable (1)/disable (0), user

**R** Right Stack Opcode bit (pending = 1)

**O** Overflow bit

**C** Carry bit

**CC** Condition Code

**Segment #** currently executing

Figure 10.9. Formats associated with code segments.

CST entry appropriately. The ADDRESS field of the CST entry is changed to become the new code segment's memory address.

Since several procedures may reside in the same code segment, several CST entries may be affected by the disk fetch operation.

In figure 10.9, the path followed by a call to an absent procedure starts with the PCAL(5), then through paths 7, 8, 9, 10 and 11.

If the procedure is present in memory (through being left around, or someone else's use of it), the CST entry so indicates this fact, and the time-consuming process of fetching the segment from disk is obviated.

Note that an external procedure STT label (i.e. a procedure not in this code segment) carries an STT number in the target code segment as well as the target segment number. Also note that the stack marker for the currently executing procedure contains the caller's segment number, and the absolute PB return location. Upon an exit, the caller's code segment might still be in memory, or might have been overwritten, and therefore must be replaced from disk. Once in memory, the return transfer is easily made by using the PB-relative return address and the CST number held in the stack marker (figure 10.7.)

Of course, upon switching code segments, the PB and PL registers associated with the program must be changed and P set appropriately.

This entire algorithm is embedded in the PCAL and EXIT instructions of the machine. The programmer need not be concerned with any of this complicated mechanism. We discuss it, because it has an important bearing on the structure of the files that a language compiler for this system should generate, the *unsegmented library* (USL) file.

### 10.5.7. Instructions

There are 13 instruction formats in this machine's architecture, six of which are illustrated in figure 10.10. We shall consider only a few of the instructions, omitting those that are of interest only to an operating systems programmer, those that are of marginal value in a compiler, and those that are essentially repetitions of those presented. We thereby hope to provide some flavor of the architecture of this machine.

Sixteen bits is a marginal instruction size for a machine of this sophistication. As we have seen, the MA (memory address) instructions have a severely limited direct address range, and there cannot be very many of them, owing to the small op-code field. Nevertheless, a large number of operations and data types can be supported through the stack architecture. Stack operations require no address field, only an operation; there can be many of them without a correspondingly large instruction field for them.

The HP-3000 stack-ops are 6 bits each and may be packed two per word. There are 64 of them, a few of which are defined next.



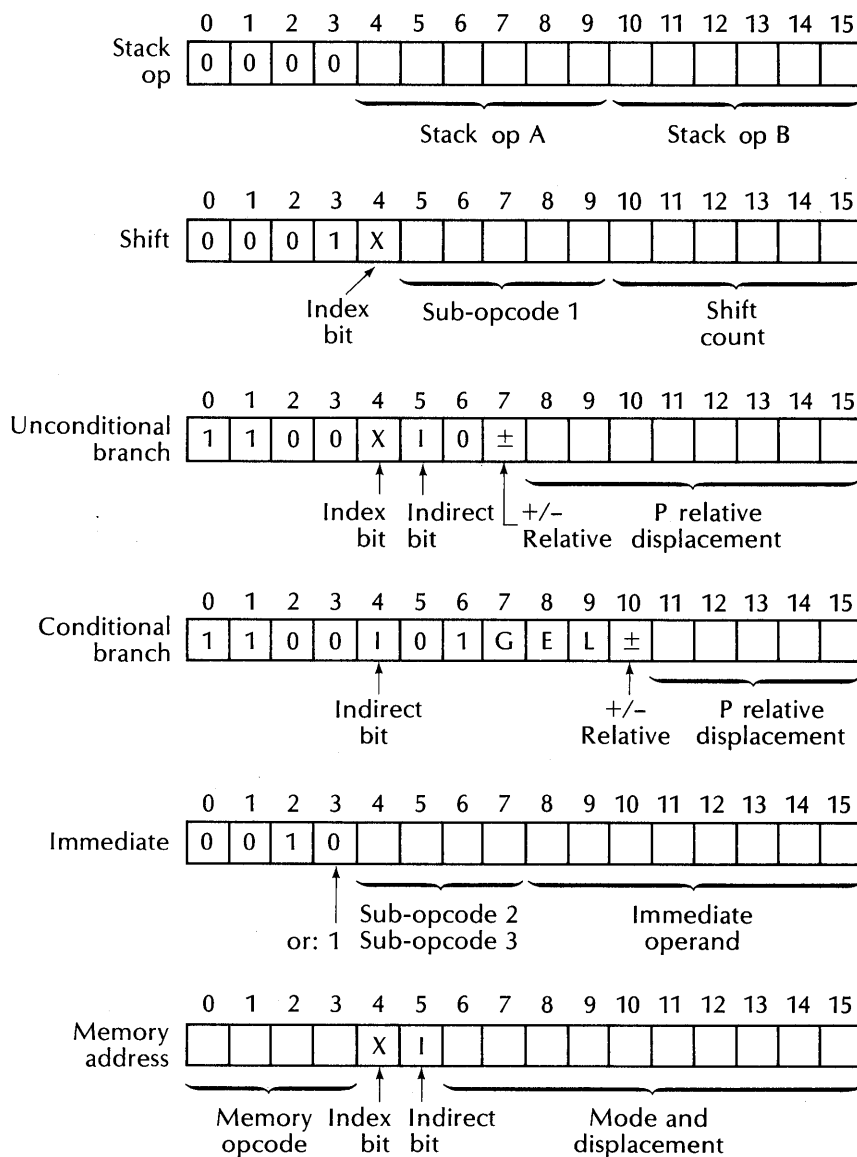


Figure 10.10. Instruction formats.

- ADD, SUB, MPY, DIV are binary 16-bit 2's complement integer operations with overflow. They of course operate on the two top-of-stack words, and replace them by one top-of-stack result word.
- LADD, LSUB, LMPY, LDIV are binary 16-bit 2's complement integer operations with rollover, but no overflow.

- DADD, DSUB are binary 32-bit 2's complement integer operations. DMPY and DDIV are available, but are not stackops.
- FADD, FSUB, FMPY, FDIV are binary 32-bit floating-point operations.
- OR, XOR, AND are binary bit-by-bit logical operations on the top two stack words.
- NOT complements the bits of the top stack word.
- NEG, DNEG, FNEG negates the TOS element—an integer, a double integer, or a floating-point value.
- FLT converts the top integer into an arithmetically equivalent floating-point 32-bit number.
- FIXR, FIXT converts the top 32-bit floating-point number to a 16-bit integer. FIXR rounds the number, and FIXT truncates the fractional part.
- CMP, LCMP, DCMP, FCMP compares the two TOS values (2 integers, 2 logical integers, 2 double integers, or 2 reals) arithmetically, deletes them, and sets the condition code CC correspondingly.  $CC=0$  means  $A > B$ ,  $CC=1$  means  $A < B$ , and  $CC=2$  means  $A = B$ , where B is in (TOS) and A is in (TOS-1).
- ZERO pushes a 1-word zero.
- DZRO pushes a 2-word zero.
- DEL deletes the top-of-stack (TOS) word.
- DDEL deletes a pair of TOS words.
- DELB deletes the word just under the TOS word.
- XCH exchanges the top two stack words.
- DUP duplicates the top word on the stack.
- DDUP duplicates the top word pair on the stack.
- CAB rotates the top three words on the stack. If they are initially A, B, C, with (TOS) = A, then CAB causes the order C, A, B.
- NOP, no operation, is used to pad a stackop to form a full instruction word.
- INCX, DECX increments (decrements) the index register.
- ZROX clears the index register to 0.
- STBX sets the index register to the word in (TOS - 1), without affecting the stack.
- STAX sets the index register to the word in TOS and deletes TOS.
- LDXA pushes the current value in the index register.

A branch to some instruction is always to a full word, hence any statement

to which a branch may occur cannot begin on an odd-position stackop. This is a reasonable restriction, but one that can be bothersome to design into a compiler.

The *immediate* instructions are listed below. Each of these carries a eight-bit field containing an operand  $n$ , where  $0 \leq n \leq 255$ .

- LDI  $n$ , push  $n$  on the stack.
- LDXI  $n$ , replace (X) by  $n$ .
- CMPI  $n$ , compare top TOS word to  $n$ , set condition code, delete TOS.
- ADDI  $n$ , add  $n$  to top TOS value.
- SUBI  $n$ , subtract  $n$  from top TOS value.
- MPYI  $n$ , multiply top TOS value by  $n$ .
- DIVI  $n$ , divide top TOS value by  $n$ .
- LDNI  $n$ , push  $-n$  on the stack.
- LDXN  $n$ , replace (X) by  $-n$ .
- CMPN  $n$ , compare TOS word to  $-n$ , set condition code, delete TOS.
- ADDS  $n$ , allocate  $n$  words on the stack.
- SUBS  $n$ , remove  $n$  words from the stack. *Note:* if  $n=0$  in the ADDS or SUBS instruction, the count is taken to be (TOS).
- ADXI  $n$ , add  $n$  to (X), replacing (X).
- SBXI  $n$ , subtract  $n$  from (X), replacing (X).
- ORI  $n$ , form logical bit-by-bit OR of  $n$  and (TOS), replacing TOS. Leading 8 bits are taken to be zero.
- ANDI  $n$ , form logical bit-by-bit AND of  $n$  and (TOS), replacing TOS.
- XORI  $n$ , form logical bit-by-bit exclusive-or of  $n$  and (TOS), replacing TOS.

These are expensive in instruction space, inasmuch as 8 bits are needed in each one for the immediate operand. However, the bulk of the literals found in common programs are small integers, and the immediate instructions obviously improve the machine's performance when they can be used. Only integer immediates are provided. All double-integer and floating-point numbers must be stored somewhere and fetched through a MA instruction. However, instructions and execution time can sometimes be reduced by using an integer immediate instruction and converting the result to the desired type.

The memory address (MA) instructions are given below. The symbol "P" means that the instruction supports P-relative addressing as well as DB, Q and S relative addressing.

- LOAD (P) fetches a word and pushes it.
- TBA, MTBA, TBX, MTBX (P) are only P-relative instructions, used to control a FOR-loop with integer index and an integer step size.

- STOR pops the TOS word to memory.
- CMPM (P) compares the TOS word to a memory word, setting the condition code and deleting TOS.
- ADDM (P) adds the memory word to TOS, result to TOS.
- SUBM (P) subtracts the memory word from TOS, result to TOS.
- MPYM (P) multiplies the memory word by TOS, result to TOS.
- INCM, DECM increments or decrements the memory word by 1. TOS is unaffected.
- LDX (P) loads X with the memory word. TOS is unaffected.
- LDB loads a byte in memory to TOS. If indirect or indexed, this expects a byte label and/or byte index offset.
- LDD (P) loads a double word to TOS.
- STB stores a byte in TOS to memory. If indirect or indexed, this expects a byte label and/or byte index offset.
- STD stores a double word on TOS to memory.
- LRA (P) generates a one-word address and pushes it. The address points to the datum specified by the address field, indirection and indexing. If P-relative, LRA generates a PB-relative word address.

Although a stack machine really only needs LOAD, STOR, and branch MA instructions, several more are provided in the HP-3000 for the sake of program efficiency. These are expensive instructions in terms of instruction bit space; the mode and displacement use 10 bits, and indexing and indirection use another 2 bits. This leaves only 4 bits for an operation code, and of these sixteen combinations, four are needed for the non-MA instructions. However, some doubling-up is possible. The STOR instructions cannot apply to P-relative addresses, so that bit 6 can be used to distinguish two store instructions. Using this trick and some others, the system designers have fit 21 distinct MA instructions in this group.

The LRA instruction is similar to the LA instruction in the AOC machine. It may accept indirection and indexing and yields a DB or PB relative address. A PB address on the stack is of limited value in this system; it is needed only for certain operating systems functions and for certain high-level vector operations that we shall not discuss.

The INCM and DECM MA instructions are valuable as optimizations. A typical program may contain several statements of the form

I:=I+1

or

I:=I-1

and these are obviously translatable to INCM I and DECM I, respectively.

The LOAD and STOR instructions have counterparts in each of the other data types, except LONG floating-point (four words). There is a LDB and STB (for bytes), a LDD and STD (for double words). Unfortunately, the long floating-point data type is not supported very effectively. Long arithmetic is performed through three addresses on the stack—result, operand 1, operand 2. This arithmetic philosophy is fundamentally different from that for the other data types and requires an elaborate compiler system to support.

The unconditional branch instruction BR (figure 10.10) can be indirect and indexed, but since indirection precedes indexing, a BR L1,I,X is essentially useless. The branch reach is limited to  $P \pm 255$ . A larger branch reach must be indirect, with the indirect word within the 255 word limit.

The conditional branch BCC (figure 10.10) is not indexable but may be indirect. The bits GEL refer to the condition code possibilities greater-than (G), equal (E), or less-than (L). The bits GEL may be set in any pattern to select one of six possible conditional branch cases (the remaining two cases yield a “never-branch” and an “always-branch” condition.) The reach of the BCC is only  $P \pm 31$ . It turns out that a fairly large number of branches are within this range of words. Those outside this range must be accessed through an indirect word.

The condition code (CC) used in the BCC instruction are set by a variety of arithmetic and stack top operations. CC is affected by some, but not all, the instructions that influence the stack top. Note that the compare operations have no other effect than setting CC. The use of a single CC register rather than the stack top is a weakness in this architecture, but one that can be surmounted if necessary; the CC and other status information can be pushed if it must be saved and later used. Recall that in the AOC machine, the result of a comparison was a TRUE or FALSE pushed on the stack.

The shift instructions (figure 10.10) accept a 6-bit shift count, and the shift count is added to (X), if the X bit is set. Single and double word shifts are provided.

The PCAL and EXIT instructions have been discussed previously. Each carries an 8-bit field, with value N ( $0 \leq N \leq 255$ ). Recall that N in the PCAL refers to a word count from the end of the code segment backwards, and that if N is zero, a procedure label is expected on the top of the stack. (It is for this reason that index 0 in the STT is not used for a procedure label.) The EXIT carries a count N ranging from 0 to 255; N is the number of words below the stack marker that are to be deleted upon the return. Thus EXIT is used to remove the formal parameters from the stack. It is the compiler's responsibility to determine N in the EXIT. However, a compiler cannot determine N in the PCAL, nor organize an STT.

The instruction LLBL causes a procedure label (copy of the STT entry) to be loaded on the stack. Like the PCAL, it refers to the STT bottom-up through a displacement N.

### 10.5.8. Allocation of Memory Space to OWN and Outer Block Variables

OWN and outer block variables in an Algol-like language belong in some data area that is unaffected by the procedure calls and exits. In the HP-3000, a reasonable static OWN area is secondary DB, just below the initial stack marker (figure 10.11). A dynamic OWN area and space for dynamically allocated arrays (like PL/I VARYING arrays) could be allocated in DL to DB; however, we shall see that there are some problems with this in this architecture.

Now the assignment of static OWN space can be made by the compiler only if every one of the procedures utilizing OWN is compiled. A fundamental premise of the HP-3000 system is that partial compilations should be provided for every language for which it is feasible. This means that any given compilation may be only of some of the procedures with OWN; consequently, the compiler will not know how to assign OWN space. This task is left to a program called a *segmenter* that does have complete procedure information.

The system also provides for additions to primary DB and secondary DB from different procedures. (However, this capability is not invoked in all the supported languages.) Hence, in general, the initial stack configuration of figure 10.11 is not determined by the compiler but through special tables intended for the segmenter.

### 10.5.9. Partial Compilation and Segmentation

Partial compilation implies another capability that must be present in the system—it should be possible for a procedure that has been previously

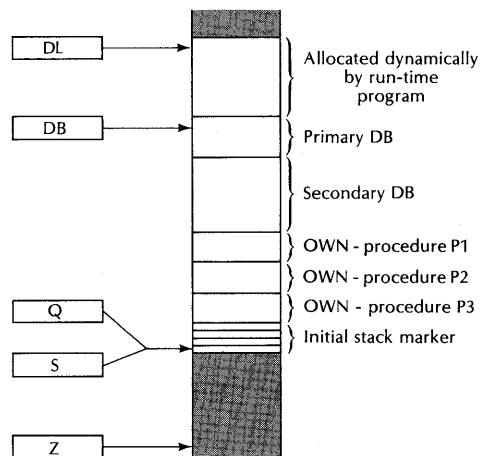


Figure 10.11. Stack configuration just before program execution.

compiled to be overwritten with a new procedure without affecting the others already written. This capability should work whether the new procedure contains more or less code, etc., than the new one. This is a useful capability, inasmuch as it significantly reduces the compilation time required to modify one or two procedures in a set of several hundred in a large program.

Another very desirable capability is arbitrary movement of procedures from one code segment to another, a process called *resegmentation*. We have seen that an external procedure call can be very expensive—a disk access may be needed, some memory manager work may be required, and lots of bookkeeping is needed. Obviously, procedures that call each other very frequently should be in the same code segment. It is desirable that considerable freedom be provided, after compilation, to move them in the interest of performance. This capability, in fact, is one of the major factors in improving execution performance on this system.

To support these capabilities, an HP-3000 compiler generates a USL (unsegmented library) file. The USL file essentially isolates procedures that are tentatively assigned to code segments but that may be reassigned. The procedures and their calls have not yet been connected. The USL file is processed by the segmenter to yield a program file. The program file is essentially a list of the code segments and the initial data segment for the program. Space for the STT entries is provided but not yet filled in; these tables can only be filled in just prior to execution. Finally, the program file is processed by the *linker* to yield an execution file. The linker is invoked just before execution. It assigns CST numbers, fixes up the STT's, and then launches the program into execution.

#### 10.5.10. The USL File Structure

The USL file consists of a set of procedure *entries* linked to *headers*, to each other, and (through a chain) to segment entries. Figure 10.12 illustrates such a file, containing three segment names, SEG1, SEG2 and SEG3, and four procedures, PROC1, PROC2, PROC3, and PROC4. (This is a simplified version of the actual HP-3000 USL file structure.)

There are four linkage systems, indicated by the directed lines in figure 10.12. The segment links start from a directory node through each of the segment entries and end in a "0" pointer (the pointers are labeled S.L.) Each segment entry must be active, and there cannot be more than one with a given name. The segment names may be defined by the compiler, or they may be defined by the segmenter.

The hash links are through each of the entries (segment or procedure), designated H.L. in the figure. These provide rapid access to any given name in the file; the segmenter must connect procedure calls (in headers) to their procedures (entries) by their names. A hash table scheme is used to locate

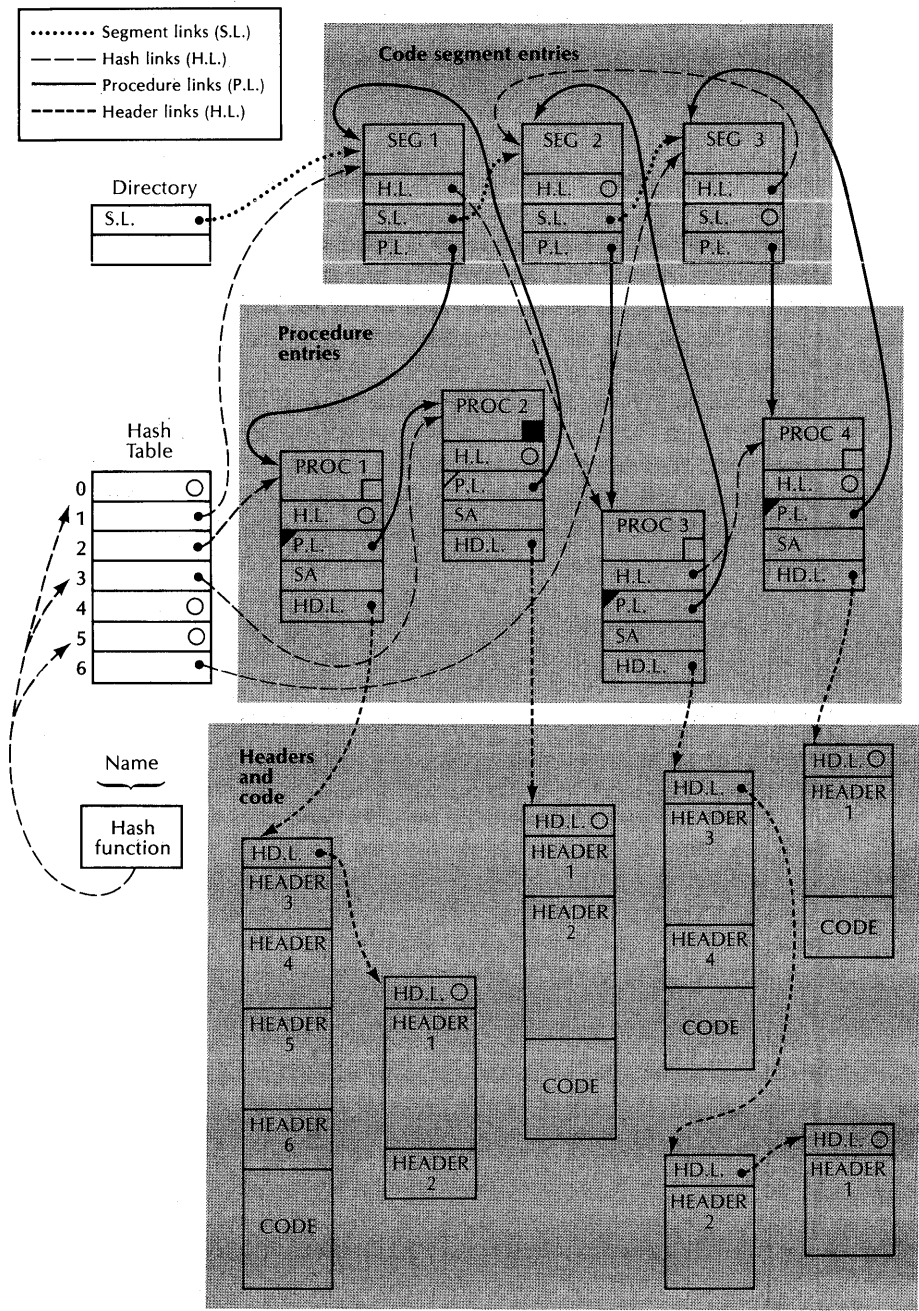


Figure 10.12. Unsegmented library (USL) file structure (simplified).



entries and segments by name; a given name is translated through a hash function into an index in the hash table. The hash chain from the table then links every name in the file with that hash code.

The third set of links are *procedure links*, designated P.L. These form a circular chain from a segment entry through every procedure entry intended to be part of that code segment. A procedure entry may be marked *inactive* (indicated by the open corner in the P.L. field), in which case it has effectively been replaced by another active one (shaded corner) somewhere. More than one procedure with the same name may exist in the USL file, but only one may be active. (The system actually permits exceptions to this rule, for the sake of having the same procedure code in several different code segments, but a discussion would take us too far afield.)

Finally, each procedure entry is linked to a set of *headers* and a *code* block. Some typical headers will be described later. They provide for certain services that the compiler cannot directly provide such as initializing outer block data space, specifying procedure calls, and fixing OWN data space. The code block is a list of the instructions in the procedure.

A header block may consist of several headers. Several header blocks may also be chained together.

It should be obvious that a procedure can be moved from one code segment to another one by merely changing its procedure links. Everything else associated with it is moved with it.

A segment entry is very simple. It only contains a name and several pointers, nothing that would require a new compilation. Hence new ones may be easily created by the segmenter.

The various entries and headers are held in one binary file, and are written as though the file consisted of a single list of words.

### 10.5.11. Procedure Compilation and USL Linkage

The compiler has a number of tasks to perform upon compiling one procedure, in addition to generating a code sequence for it, as follows (see figure 10.12).

1. Code and headers must be written to the USL file and their location noted. In general, new code and headers are simply added to an existing file by appending them.
2. At the end of a procedure, the hash chain for that procedure is searched for any existing procedures with that name. Any existing ones are marked inactive, but no attempt need be made to reclaim their space.
3. A new entry for the procedure is written to the file and connected to the hash chain at its head. Note that this means that the most recently

entered ones are the first noted by the segmenter in its search for procedure names. The hash table is then modified and updated.

4. The new procedure entry is linked into its procedure chain, again at the head of the chain. For this, the compiler must know the segment in which the new procedure belongs.
5. The headers belonging to the procedure are linked to the procedure entry.
6. The directory contains certain information regarding the available and unused space in the USL file; this information must be updated by rewriting the directory.

The compiler needs the following tables in memory:

- SEG: An image of the current segment entry.
- DIR: An image of the current directory.
- HASH: An image of the current hash table.
- PROC: An image of the current procedure entry.
- HEADER BUFFER: A one-record buffer for headers and code.

The header buffer is essentially filled as headers or code are generated (only one or the other at a time); when full, it is emitted to the USL file as a complete record. At the end of a procedure, it must be flushed to disk. The DIR and PROC tables are of course updated to reflect the written header and code locations and lengths.

At a procedure end, PROC is written to disk and linked into its chains as previously described. SEG is then updated on disk, by either replacing an existing one or adding a new one into the seg and hash links. DIR and HASH are then written to disk; these overwrite the previous ones.

By only updating the USL file in this manner at a procedure end, the USL file is protected against some program error or other serious malfunction in the compiler. There shouldn't be any, but sometimes a user will abort the compiler without thinking about its effect on the USL file. If the compiler is aborted, or a procedure error is detected, no harm is done. Although header information has been written, the file directory on disk is not aware of it, and all the file links reflect a configuration in which the failing procedure has not even been seen.

### 10.5.12. Primary DB Header

We now turn to the structure of certain of the USL headers. Their form is rather arbitrary, but reflects a solution to the segmentation requirements stated previously.

Every header has a common first word structure containing NW, the

number of words in the header, and a header type code. From these, the length and the type of each header are known. (Note from figure 10.12 that headers can either be arranged in a contiguous list or linked. We have not shown certain details such as how the net length of a list is determined.)

The primary DB header, figure 10.13, specifies the arrangement of the first 255 words (or less) of DB. These may be absolute values or pointers into secondary DB. Since the segmenter, not the compiler, fixes the location of secondary DB, the values of the pointers cannot be set by the compiler; the compiler can only set the relative offset into the secondary DB area.

The segmenter must be told which of the primary DB cells are absolute values and which are pointers; that is the purpose of the array  $U_i$ ,  $0 \leq i \leq N-1$ , where  $N$  primary words are specified. The value  $N$  can be inferred from the number of words in the block  $NW$ .

If  $U_i = 0$ , then  $C(i)$  is unchanged; this would be an absolute value. If  $U_i = 2$ , then  $C(i)$  must be incremented by  $SDBA$  by the segmenter;  $SDBA$  is the DB-relative origin of secondary DB.

The size of this header block is variable, but cannot be greater than 289 words because of the 256-word limit of primary DB.

#### 10.5.13. Secondary DB/OWN Initial Values Header

Initial values for secondary DB or OWN can be specified by the header in figure 10.14. Of course, some languages do not provide initialization as part

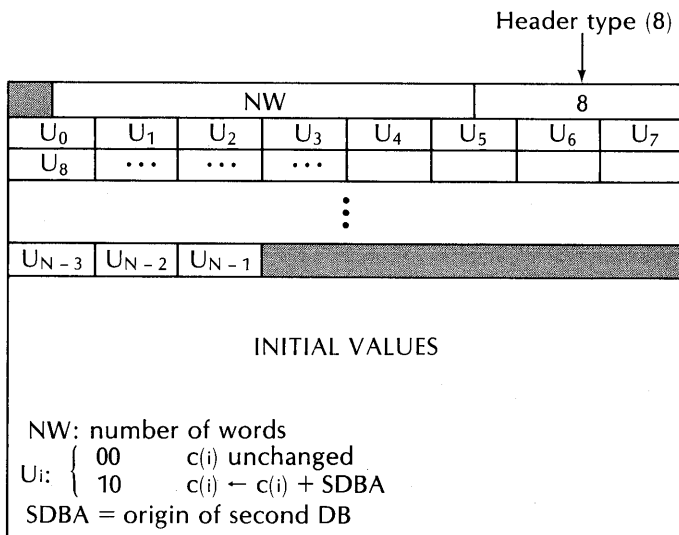
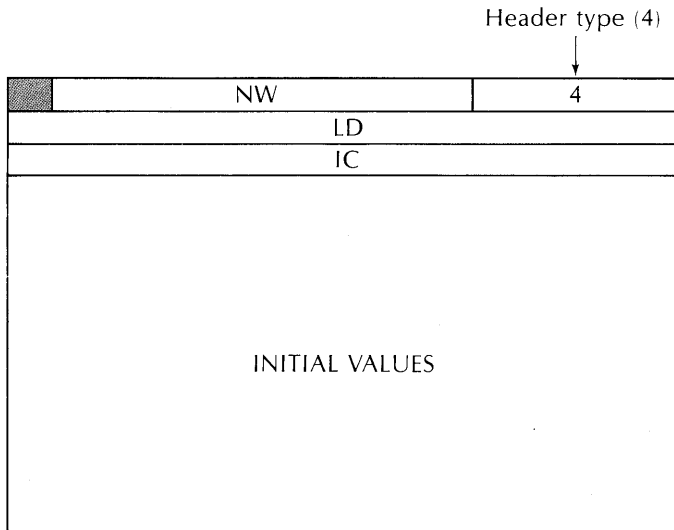


Figure 10.13. Primary DB header.



**Figure 10.14.** Secondary DB and OWN initial values header.

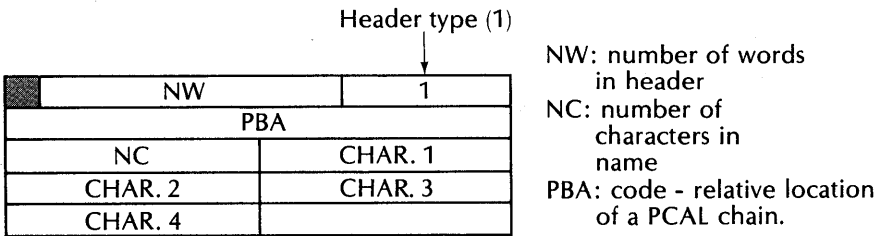
of a declaration; in these, this header would be unnecessary. Also, in principle, the compiler could generate special code executed prior to user code, to initialize the desired data areas. However, the initialization information must be stored somewhere, and it might as well be stored in the USL file as in code.

Parameter LD is a word displacement of the desired initialization data in the secondary DB or OWN area, regarding the first of the area as having displacement 0. The segmenter must then write the initialization words in  $DB + LD + SDBA$ , for DB, or  $DB + LD + OWNBA$ , for an OWN. SDBA is the DB-relative address of secondary DB. OWNBA is the base address of an OWN area assigned to this procedure.

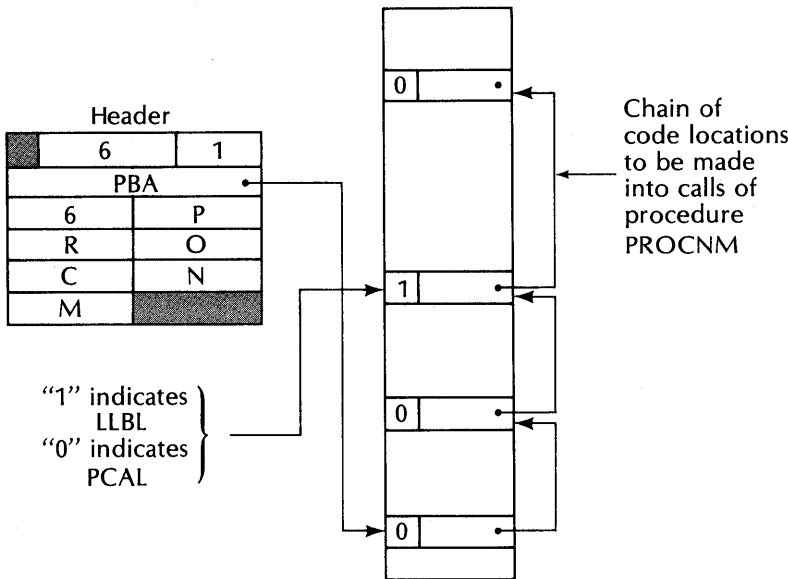
IC is a repetition factor, and it is useful for initializing large arrays with a repetitive sequence of values. For example, a large array might be initialized to zeroes by setting IC to the array size and placing one 0 in the “initial values” section.

#### 10.5.14. Procedure Call Header

Figure 10.15 illustrates a procedure call header and its application. As we have seen, a procedure is called through a PCAL instruction which expects a STT index number as its operand. However, the STT is organized by the segmenter, not the compiler. The STT's will of course change as procedures are moved around among the segments after compilation, so that the compiler has no idea where the procedures will be located, or the STT index for the PCAL instruction.



(a) Procedure call header



(b) Use of call header

Figure 10.15. Procedure call header and its application.

The PCAL header contains a code-relative index PBA and the procedure name. PBA points to a code location that must receive a PCAL (or LLBL, depending on the sign bit setting). When the compiler writes the PCAL, it places a pointer to another PCAL location. These therefore form a chain pointing backward through the code, linking every PCAL and LLBL associated with one procedure (figure 10.15(b)).

When the file is processed by the segmenter, each word on this chain is replaced by the appropriate PCAL or LLBL instruction to form a completed procedure code list.

Of course, the segmenter locates the procedures through the USL file hash system.

### 10.5.15. OWN Variable Pointer Correction Header

Figure 10.16 shows a header that deals with another problem related to OWN variables. Given that the location of OWN variables is fixed by the segmenter, something in the run-time code must nevertheless be adjusted to reflect the final location of some OWN variable. Now OWN variables are accessed indirectly through a label that is written on the stack just after a procedure call. The label is placed on the stack by copying some program location; this location must therefore be corrected by the segmenter, once it decides on the location of the OWN data area for that procedure.

PBA is the location in CODE of a word that must be offset by the location of the OWN data area as fixed by the segmenter. See section 10.11 and the discussion of section 10.5.8 for more details.

The code location PBA is therefore corrected by the segmenter as follows:

$$\text{CODE(PBA)} := \text{CODE(PBA)} + \text{OWNBA}$$

where OWNBA is the DB-relative OWN base address for this procedure. Clearly, the compiler must write an offset of the variable in CODE(PBA).

### 10.5.16. Procedure Local Variables

As figure 10.3 shows, the direct addressing range for  $Q+n$  addresses is limited to  $n < 128$  words. By using indirection judiciously, the number of words of local variables can be much greater than this.

Here is a simple *local variable allocation rule*.

1. Each simple single-word variable is assigned a direct access cell.
2. Each array variable and each multiword simple variable is assigned one direct access cell to be used as an address, and as many indirect access words (above Q) as needed.

This rule permits a procedure to have as many as 127 distinct local variables, but they may carry more than 127 words total.

Since indirect addresses are relative to DB, not to themselves or Q, and the location of Q will change from one call to the next, the addresses of local arrays will have to be constructed during execution upon each procedure call. The local variables are efficiently set up as follows:

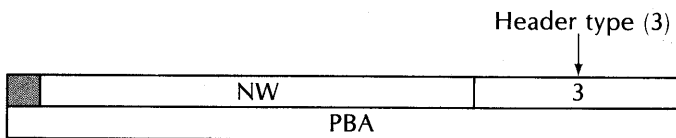


Figure 10.16. OWN variable pointer correction.

1. Upon completion of the PCAL for the procedure, Q and S point to the same stack word, the top stack marker word. If there are any array variables or multiword simple variables, then the instruction

LRA Q+n

is emitted, where n is the Q-relative word address of the first variable. The value of n will be known after all the local variables have been scanned. This writes a DB-relative address label on the stack top.

2. For each array variable or multiword variable containing w words, the instructions

DUP  
ADDI w {or ADDM =w}

are emitted. These instructions provide another address on the stack, pointing w words past the previous one.

3. If there are m words of simple variables that have not yet been allocated, and p words of arrays and multiword variables, then

ADDS m+p {or LOAD m+p; ADDS 0}

is emitted.

This construction process can be expanded to provide for initialization of some or all of the data spaces. Initialization values are best kept in the code segment of the called procedure and moved to the stack.

Nothing need be done about the local variables upon an exit, as the EXIT instruction effectively deletes all the local variables, the stack marker, and the formal parameters by resetting S.

OWN variables are somewhat more bother. OWN variables must be allocated in secondary DB (see figure 10.11). The most efficient approach is to allocate primary DB space for simple variables or addresses of multiword variables. These DB addresses are only known within the procedure containing the OWN, of course. Unfortunately, primary DB space is limited to 255 words, and this places a severe restriction on the maximum size of programs that can be so supported.

A less efficient approach, but one that conserves primary DB, is to write the OWN addresses on the stack upon entering a procedure. These addresses are not known to the compiler if partial compilation is permitted, hence must be assigned by the segmenter. The scheme makes use of the *OWN variable*

*pointer correction header* (figure 10.16.) This USL header specifies a code segment location that is to be adjusted to the secondary DB address of some OWN variable. When the program is executed and the address of the OWN variable is needed on the stack, the instruction

LOAD P-m      {or LOAD P-m,I}

suffices to load an OWN address. Here,  $m$  is the displacement of the code segment address relative to the LOAD instruction. (If the displacement  $m > 255$ , then an indirect PB label must be used to fetch the OWN word.)

### 10.5.17. Branches and Constants

Branches have a limited range because of the small number of bits in the partial address field (see figure 10.10). The maximum range is  $P \pm n$ , where  $n = 255$  for an unconditional branch and  $n = 31$  for a conditional branch, and where  $P$  is the location of the branch instruction. If a branch to an instruction past this range must be coded, then an indirect address word is needed in code; this word must not be executed as an instruction, of course.

Constants are also conveniently placed in a code segment. Three advantages accrue thereby—the constant values cannot be inadvertently altered through a program error, they are always available (within the same segment as the instruction), and they do not use up valuable DB addresses. Most of the load instructions can be used for PB-relative words, however, the range is limited to  $\pm 255$  words relative to the instruction.

A disadvantage in locating constants in code on this machine is that they cannot be passed by reference to a procedure; their address label in a code segment is indistinguishable from a stack address label to the called procedure. The constant must first be loaded on the stack, then its stack address passed. It might as well be given a stack location to begin with in such a case.

We clearly need an efficient algorithm for the allocation of PB space to instructions and *noninstructions* (constants or indirect words). A sketch of a scheme that yields optimal code allocation follows.

We note first that because noninstructions will in general interrupt an instruction sequence and will require a branch around them, we should (1) accumulate a block of noninstructions, to minimize the number of branch-arounds, (2) accumulate no more than 255 words in order to code a direct branch-around, (3) exploit both  $P+$  and  $P-$  addressing, and (4) minimize the number of indirect words required.

The process requires two passes. On the first pass, two lists are prepared, a CODE list, consisting only of executable instructions, and a fixup list FX, consisting of blocks of two words each as follows (see figure 10.17)

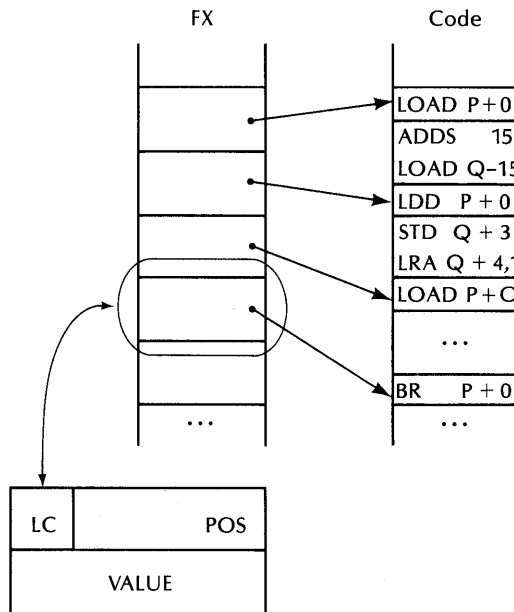


LC: 1 if instruction refers to an instruction label.  
       0 if instruction refers to a constant.  
 POS: position of instruction in CODE list.  
 VALUE: the constant value if LC=0, or  
       the position of the  
       target instruction if LC=1.

Every P-relative instruction in CODE is assigned an FX entry; the instruction itself with an empty address field is placed in CODE. The FX entries are in the same order as the CODE entries.

At the end of the first pass, we have completed a procedure. Every branch target location is therefore known, and the FX table will therefore be complete.

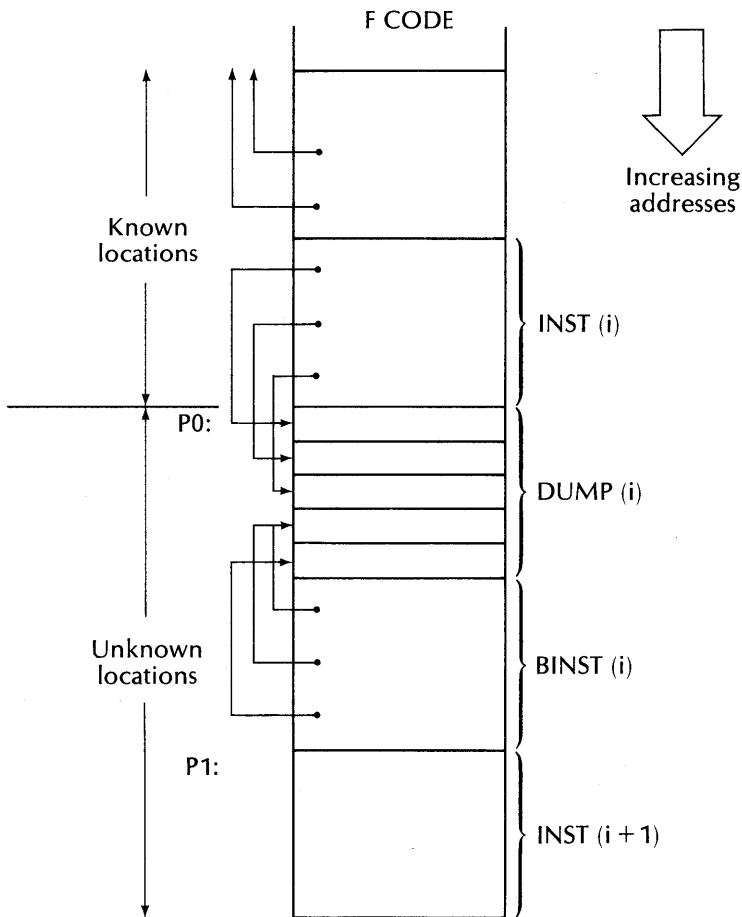
The second pass is through the FX-CODE list. On this pass, the CODE list will be partitioned into blocks of at least 510 words each. (We assume that all instruction reaches are 255 words, for simplicity.) Between each instruction block will be a dump area DUMP of constants and indirect cells. The general idea is that every P-relative instruction will be able to either reach its target directly or through an indirect cell in a neighboring dump area ahead of or behind the instruction. The target may be another instruction or a dump area word.



**Figure 10.17.** Code and fixup list, FX. FX contains an entry for every P-relative instruction in CODE.

The partitioning, assignment of dump words, and fixup of instructions will occur in one general fixup operation. The nature of the operation is suggested by figure 10.18. At a critical point in the fixup, a list of fixed-up instructions and dumps are in a list FCODE, preceding P0. The locations of the instructions up to address P0 are fixed and known, however, a block of instructions INST(i) preceding P0 remains to be fixed up, a dump area DUMP(i) has yet to be determined, and there is some group of instructions BINST(i) whose position is unknown.

The first fixup task, given the situation of figure 10.18, is to determine the size of the DUMP(i) area. We do this by scanning the region of FX that corresponds to INST(i) and BINST(i). Since P0 is fixed and known, the size



**Figure 10.18.** Configuration upon beginning to fix DUMP(i) area. We look for constants and instructions that require an indirect word within regions INST(i) and BINST(i); these will be placed in DUMP(i).

of  $BINST(i)$  will be 255 words; no instruction located farther than 255 words past  $DUMP(i)$  can reach  $DUMP(i)$  directly, hence is irrelevant.

Words are placed in  $DUMP(i)$  in the order in which they are encountered in the FX table for the FCODE block  $i$ . This ordering facilitates the reach situation, since the first instructions in  $INST(i)$  will access the first words in  $DUMP(i)$ , and the last instructions in  $BINST(i)$  will access the last words in  $DUMP(i)$ .

Every constant must be placed in  $DUMP$ , however, a multiply referenced constant need only be placed once, provided that each of the references can reach it. Since shared constants are rare, a reasonable plan is to enter every constant as encountered to establish an initial size for  $DUMP$ ,  $D$ .

We next add those indirect words that are essential for branching. To do this, we identify a branch instruction with the maximum reach (given the current  $DUMP$  size), which can be achieved through a scan of the FX table for region  $i$ . That instruction can either be fixed up with a direct reference or will require a  $DUMP$  cell. If it can be fixed up, then all the other instructions can also be fixed up, and no more  $DUMP$  words will be needed. If the instruction cannot be directly fixed up, then it needs a  $DUMP$  word, and the next largest reach must be found. Instructions that can share an indirect  $DUMP$  word with a preceding one can be ignored.

In searching FX for a maximum reach instruction, every location  $LOC > P_0$  and position  $POS \geq P_0$  must be augmented by the current  $DUMP$  size  $D$ ; the required reach  $R$  is clearly

$$R := \text{ABS}((\text{if } LOC > P_0 \text{ then } LOC + D \text{ else } LOC) - (\text{if } POS > P_0 \text{ then } POS + D \text{ else } POS))$$

If  $R > 255$ , and an existing word in  $DUMP$  cannot be used, then a new  $DUMP$  word is needed:  $D := D + 1$ .

Once the  $DUMP(i)$  area is known, the instructions in  $INST(i)$  can be fixed up from FX information, the  $DUMP(i)$  area can be emitted, and the  $BINST(i)$  instructions can be located and fixed up; these will take us as far as location  $P_1$ . The remaining FX table must also be adjusted to account for the  $DUMP$  area size; this is done by adding the dump size  $D$  to every  $VALUE$  and  $POS$  found greater than or equal to  $P_0$ .

We now must establish the next  $P_0$  position. We start with the next FX entry not yet dealt with. If it is a forward or reverse branch with a target within its reach, the instruction is fixed up and we continue the FX table scan. Otherwise, the instruction will require a  $DUMP$  word, and we therefore use it as the origin of the  $INST(i + 1)$  area. The end of the  $INST(i + 1)$  area,  $P_0$ , is 255 words farther on, and the position  $P_1$  is 511 words farther on. (Some allowance must be made for the branch around the dump area). We are then in a position to repeat the above process.

We see that the overall effect of this algorithm is the clustering of dump

blocks, minimizing their size, and maximizing their separation. The algorithm becomes considerably more complicated if the two different P-relative instruction reaches (31 and 255) are both taken into account. We have also assumed that every constant is a single word. Multiword constants are probably best left to the end of the code and accessed indirectly; however, this is a suboptimal approach in general. Multiword constants could conceivably be placed in dump areas, but the fixup algorithm would become more complex.

### 10.5.18. Conclusions

It should be apparent that a compiler for this machine must do much more than simply generate a sequence of stack operations for expressions and assignment statements, that is, if partial compilation or resegmentation after compilation is deemed desirable. If neither of these is wanted, then the compiler could generate an absolute loader file and perform its own procedure linkage, STT generation, etc.

In particular, the compiler must generate the USL file structure illustrated in figure 10.12, and be prepared to increment an existing USL file with new information. Certain problems exist with this USL file that introduce compiler difficulties as follows:

1. CODE should be partitioned among several linked blocks rather than appear in one contiguous block in the file. Now it happens that during compilation, code and headers will have to be emitted in an intermixed fashion in general. This means that one or the other must be written to a temporary file or memory area and later copied to the USL file.
2. The initialization header (figure 10.13) has a maximum NW of 1023 words because of its 10 bit field. However, the system can support many more words in a DB or OWN array (up to the limit of 32,000 total for the stack). Therefore, the initialization header generator must be prepared to partition an initialization block for some array into pieces of 1023 words each at most. Furthermore, even a 1023 word header will either require a large memory buffer or must be constructed "on the fly" by sending individual words to a general-purpose header writer.
3. Only one code segment with a given name is permitted on the segment chain, yet several procedures with the same name can appear on the procedure chain. This means that these two entries must be handled in a different fashion. It would be better to treat code segments exactly the same as procedure segments.

These are obviously minor design complaints, and they might have been corrected when the USL structure was designed. However, this structure is

used by several language compilers, and it is impractical to change it. The key point is that the compiler designer must clearly understand the USL rules and provide for its design quirks.

Upon examining the 3000 machine architecture, we find a number of other quibbles that force complications on the compilers as follows:

1. The HP-3000 instruction set is incomplete. For example, memory-reference arithmetic instructions are provided for ADD, SUB and MPY, but not DIV. Also, these instructions are applicable only to integer arithmetic. An optimization to take advantage of them clearly must be accompanied by tests for these conditions. As another example, although long data types are defined and provided for through stack-op instructions (not given here), the instructions expect three addresses on the stack top. This convention is fundamentally different from that of the other data types and obviously requires a special compiler algorithm to support. For one thing, the problem of temporaries crops up. It might be best to simulate the stack-value approach by first allocating stack top space for the two variables, then load their addresses, then execute the instruction. The conversion instructions are also incomplete; however, any conversion can be made with a combination of one, two, or three stackop instructions.
2. The architecture requires three kinds of address labels: a data memory word label, a data memory byte label, and a PB-relative code label. A word label carries a DB-relative word position, a byte label carries a DB-relative byte position, and a PB-relative label carries a PB-relative word position. These are each carried in one 16-bit word and cannot be distinguished at run-time. They can only be distinguished at compile time. Furthermore, a PB-relative label is of no use when passed to a procedure in a different code segment, as no mechanism for accessing the words of the caller's code segment exists, and, of course, the caller's code segment number is lost. The two kinds of data labels, along with the need to organize some arrays through indirect labels, greatly complicates the handling of equivalences. For example, consider an array passed by reference to a procedure. If the actual parameter is a byte array, then the formal parameter should be, too, and a byte label must be passed. Otherwise, a word label should be passed. Clearly, full knowledge of the formal parameter declarations is needed when the procedure call is coded.
3. Only one Q register is available. As we have seen in chapter 9, it would be desirable to have a display for block-structured languages. However, the missing display can easily be compensated for in this architecture. We have two options in dealing with up-level addressing: follow the

static chain to obtain each reference, or follow the chain and write address labels in the local data space. The former is less efficient, while the latter requires two passes of a procedure, one to identify all the up-level calls in the main body of the procedure and a second to generate the code for the necessary address labels.

4. Only one level of indirection is provided. This is a serious handicap in dealing with dynamic arrays. As we have seen in chapter 9, two indirections are needed for the management of a dynamic array space. One might suppose that the following code sequence could be used for a dynamic array address fetch:

```
LOAD  Q+n,I {load a label}
LDD   S,I,X {then load the variable—a double word}
```

However, this operation now yields a label on the stack underneath the variable that must be removed. That is not too bad; the instructions

CAB, DEL

achieve that. The real problem is that it would be desirable to be able to pass dynamic arrays by reference to procedures just as nondynamic arrays are passed. The called procedure needs one uniform way of dealing with them, and this must of necessity be a double indirection for every such array which is clearly inefficient for nondynamic arrays. The only effective solution to this problem is a tag bit on indirect words that effectively says, "I am pointing to another label". This tag is lacking in the 3000, and would be impossible for the manufacturer to add as an architectural extension.

5. Different instructions have different direct address spans. (There are four different effective spans, 255, 127, 63, and 31.) These differences complicate the compiler systems that must keep track of direct addressing limits. The two different P-relative spans (31 and 255) complicate the compiler algorithms that must determine how to place indirect words among the code.
6. The restriction that non-byte variables must be aligned on word boundaries is a problem in such languages as Cobol and Fortran that permit a more general alignment. In Cobol the alignment problems arise through file access; the language permits detailed specification of the location of variables in a file record, and this specification can require alignment of a double-word data variable on an odd byte boundary, for example. In Fortran, alignment problems arise through the EQUIVALENCE statement.

7. The very small number of directly addressable cells available means that special techniques are required to support large programs. For example, many Fortran subprograms contain more than 127 variables. Such programs could not be supported on this architecture if the rule is "one variable per primary word." A way around this limitation is to allocate primary words for variables and pointers, until only one primary word is left. It then is a pointer to an array containing all the remaining data. The compiler must keep track of the offset of the remaining data in this array and use the index register to access data.
8. The code segment size limitation is a restriction on the size of the largest procedure or main program that can be compiled. This restriction can only be lifted by partitioning a large procedure into several smaller ones. Although such a partitioning is conceivably possible automatically through a data flow analysis, in practice only the program writer can do it effectively.

Despite these technical difficulties, it is relatively easy to design a compiler for this architecture. The stack-based arithmetic and procedure control system, and the separation of code and data segments facilitate block-structured languages with recursive procedure calls. Indeed, the systems programming language, designed to access all the machine features efficiently, is a modified version of Algol 60.

### Exercises

1. What determines the maximum size of a code segment, other than the stated operating system constraint? What determines the maximum size of a data segment?
2. A procedure may be arbitrarily located within a code segment. Only procedure calls PCAL and load label instructions LLBL need be adjusted by the segmenter in the process. Explain why.
3. Why is a data indirect word DB-relative, rather than Q- or S-relative? (Hint—consider a procedure parameter passed by reference.)
4. Describe how parameters can be passed by value, by reference, and by name in a procedure call. How can a procedure name be passed and later called within the called procedure?
5. Given an Pascal-type procedure declaration, sketch an algorithm that assigns addresses to the formal and local variables.
6. Design an HP-3000 code sequence that writes an up-level address on the top of stack. This code could be used in addition to the local variable stack setup upon entering a procedure.

7. Write a procedure that walks through a binary tree located somewhere in DL to S. The procedure may be recursive. Each tree node consists of two words representing the DB-relative address of the left and the right son node, respectively. A word containing octal 100,000 marks a leaf node. First write the procedure in Algol or Pascal notation, then translate it into HP-3000 assembly language.
8. Indirect words may refer either to bytes or to words. A byte label is the position in bytes relative to the left-most byte in DB0. Given that local declarations may contain both word data types and byte data types, sketch an algorithm that efficiently organizes the stack for the local variables of a procedure. Assume that the local declarations are either simple integers, bytes, fixed integer arrays or fixed byte arrays.
9. Pascal dynamic structures can be organized in DL to DB space on the HP-3000 without the use of double indirection by the following scheme. Each structure is of fixed size, since it is associated with some data type; it is linked under program control to others through pointers saved in the structure. The compiler should then make a list of the different word sizes needed for the dynamic types found in the program, and organize a linked list for each size. Each list will contain some "used" and some "spare" entries in general. A new request can be serviced by allocating one of the spare entries or by allocating a new list element. An entry can be discarded by returning it to the "spare" status. In this way, garbage collection is never needed, and the entries need never be moved, once activated. Sketch an algorithm to support such dynamic structures.
10. Construct a flowchart or design a set of procedures that support procedure calls and exits on the HP-3000. Define the necessary services required of the memory manager and disk file manager.

### 10.6. The Control Data 6000 Computer System

The Control Data Corporation (CDC) 6000 series is a large, general purpose, data processing system, intended particularly for scientific computing. A major emphasis in its design was placed on fast floating-point arithmetic, almost to the exclusion of string and decimal operations. The Cybernet 7600 systems are also based on the 6000 architecture, but contain a set of string and decimal instructions.

This particular system is of interest for two reasons: (1) the central processor architecture is a good example of a multiregister system with a relatively simple instruction set, and (2) Nicklaus Wirth found that Pascal was fairly easy to implement with the 6000 registers and operations.



The 6000 system consists of three major components—a *central processor* (CP) unit (two in a 6500 system); a large, fast *central memory*; and ten identical *peripheral processor units* (PP's.) Each of the peripheral processors has its own small memory and special instruction set. The central processor has direct access only to central memory. It communicates with the PP's through messages left in central memory, otherwise it and the PP's operate asynchronously during program execution.

The PP's can communicate with the system's data channels and main memory. They can also cause the CP to switch tasks through a special instruction called an *exchange jump*. The PP's are only used for operating systems function, and the CP is essentially only used for user program execution. However, some operating system functions are performed by the CP as special tasks, and all input-output operations must pass through the PP's.

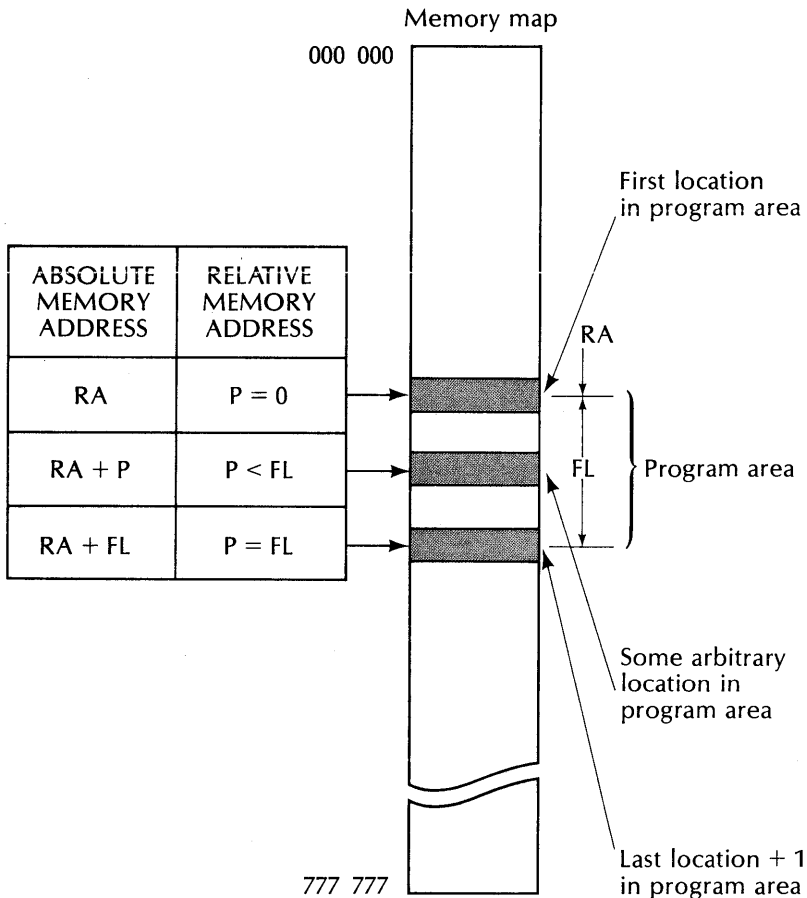
This system is intended for multitasking and multiprogramming. Main memory consists of 60-bit words, with 131,072 words maximum in a system. It may be partitioned into a number of independent contiguous program areas by the operating system memory manager. Not all of the memory is available for a user, however. Some is preempted for the operating system. Data and instructions share the same program space, although a language compiler may of course distinguish the two through software conventions.

Each partition of main memory is controlled through two machine registers, RA and FL (figure 10.19.) RA fixes the base of a program area; every memory reference instruction address is relative to RA. FL fixes the field length. Any attempted access of main memory below RA (possible through an address roll-over) or above RA + FL results in a program abort. FL may be adjusted dynamically through an operating system procedure call.

During time-shared operation of the system, a user program area may be moved to and from mass storage (disk) under control of the operating system. Since there is no distinction between fixed instruction memory and variable data memory, all of the program area is assumed variable, and must be written to disk whenever the memory manager decides to put that job to rest for a while.

An overlay facility is also provided by the operating system to accommodate programs larger than 131,000 words, or to partition a large program into smaller program blocks. However, it is only semiautomatic and requires careful planning on the part of its users.

The central processing unit consists of a set of 24 data registers, a set of one or more instruction registers, and an arithmetic section (figure 10.20). The 6400 and 6500 systems have one instruction register and one arithmetic unit; the 6600 system has an 8-word instruction register queue and ten arithmetic functional units. The 6600 was designed to improve CP performance through parallel processing of arithmetic operations and through providing several instructions in a fast register set. Otherwise, the instruction sets of the three 6000 CPs are identical.



**Figure 10.19.** CDC6000 main memory organization. Area for one process is RA through RA+FL.

### 10.6.1. Registers and Arithmetic

An arithmetic or logical operation can only be performed on an operand (or operands) in registers (figure 10.20.) Most of the arithmetic operations use the eight X registers, 60 bits each. The A and B registers are principally used for simple address calculations.

Only full addresses (18 bits) are used in instructions; there are no partial addresses to contend with.

The A registers are coupled to the X registers in an interesting way. Any value deposited in register  $A_i$ , where  $1 \leq i \leq 5$ , causes the contents of memory location  $A_i$  to be copied into register  $X_i$ . Any address deposited in register  $A_j$ , where  $6 \leq j \leq 7$ , causes the contents of register  $X_j$  to be written to memory location  $A_j$ . (The new value of  $A_i$  or  $A_j$  is used as the memory address).

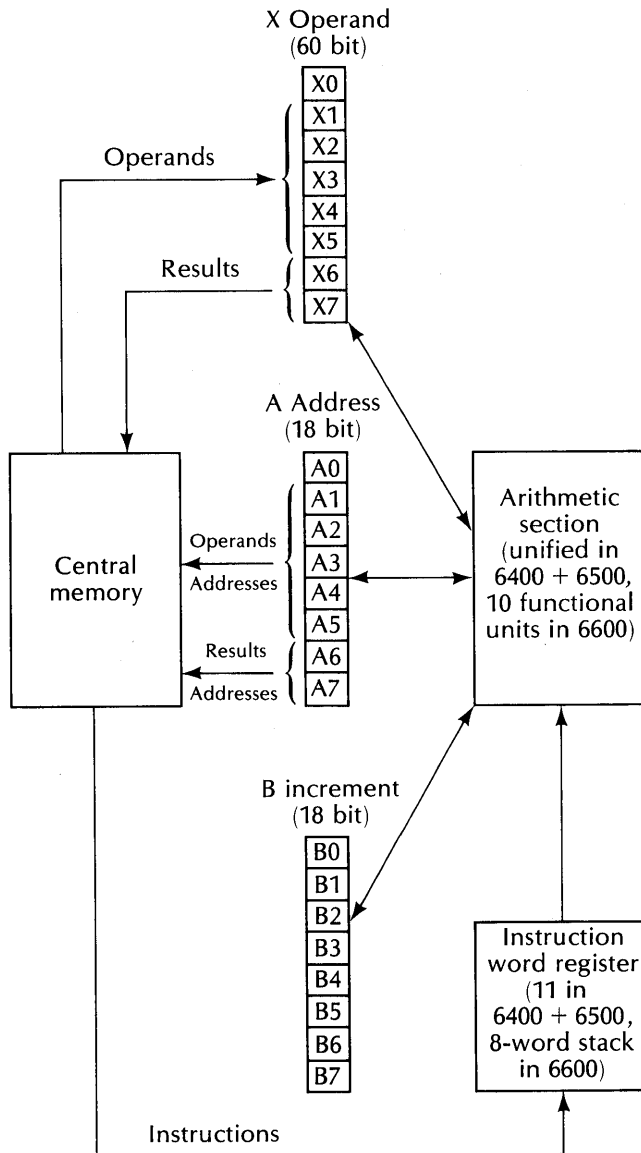


Figure 10.20. CDC6000 register and memory organization.

This X-register loading feature is the principal means the CP has of communicating with main memory.

Register B0 contains zero permanently. Any value stored in B0 disappears, and any reference to B0 is equivalent to a reference to zero.

Registers A0 and X0 are unconnected; no memory access with X0 occurs

when A0 is loaded. A0 can carry any 18-bit value, and X0 any 60-bit value.

The B registers play a prominent role as address offsets in the instructions. They are essentially index registers.

### 10.6.2. Instruction Format

The CP instructions are either 15 or 30 bits in length. They are normally packed 2, 3, or 4 to a 60-bit word in memory. A NOP instruction (15 bits) is provided for padding, when full packing is impossible. See figure 10.21.

Branches are to the left-most instruction of a 60-bit full word, not to some 15- or 30-bit internal boundary. For this reason, the compilers and assemblers must be sure that a branch target is the left-most instruction in a word.

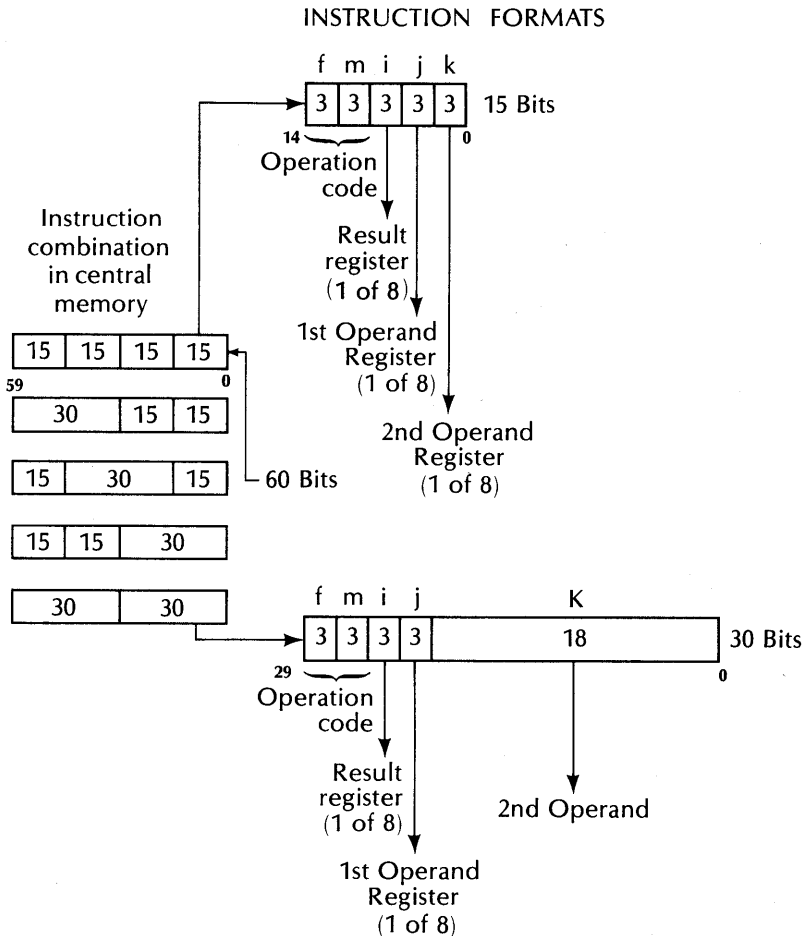


Figure 10.21. CDC6000 instruction formats.

The 15-bit instruction format carries five fields of 3 bits each,  $f$ ,  $m$ ,  $i$ ,  $j$ ,  $k$ . Fields  $f$  and  $m$  comprise a 6-bit operation code, and fields  $i$ ,  $j$ , and  $k$  are register indices in general (there are certain exceptions). The most common 15-bit instruction is a binary operation on two registers  $j$  and  $k$ , with the result appearing in register  $i$ .

The 30-bit instruction format carries an operation code (6 bits), a result register index (3 bits), an operand register index (3 bits), and a full memory operand address  $K$  (18 bits).

Every instruction with a “K” reference is 30 bits; all other instructions are 15 bits.

The registers are fully symmetrical, aside from the special A register feature that also accesses memory. Any two registers (whether the same or different registers) can be the operands, and the result can be returned in any of the operand registers. The system copies the source register contents into temporary work registers to perform the necessary operations and then writes the result to the destination register.

### 10.6.3. The Register Set Operations

The general format of a register *set* operation is below. There are two forms, an “S” form and a “B” form. (COMPASS mnemonics will be used in this discussion, to avoid entanglement with octal instruction codes. COMPASS is the 6000-series assembler.)

$$S \left\{ \begin{matrix} A_i \\ B_i \\ X_i \end{matrix} \right\} \left\{ \begin{matrix} A_j \\ B_j \\ X_j \end{matrix} \right\} + \left\{ \begin{matrix} B_k & (15 \text{ bit}) \\ K & (30 \text{ bit}) \end{matrix} \right\}$$

BXi Xj

The S form can be used to set any of the A, B or X registers with a sum or difference of  $\{A, B, X\}$  and  $\{B, K\}$ . The S form instructions are complete with one exception:  $X_j - B_k$  is not provided. Recall that  $K$  designates an arbitrary 18-bit address, relative to the program origin RA. Also recall that any  $SA_i$  (for  $1 \leq i \leq 7$ ) carries a memory access side effect, affecting register  $X_i$  or a memory word.

The B form copies one X register to another, e.g.,

BX3 X4

copies register X4's contents to register X3; X4 is unaffected. When an X register is set with an S-form instruction, only the least significant 18 bits are set; the most significant bits are zeroed.

Some examples of set instructions are:

SB1 B3+B4 {sum of C(B3), C(B4)  
replaces C(B1)}  
SA2 B2-750 {C(B2)-750 replaces  
replaces C(A2); then C(X2)  
is written to memory address  
C(A2)}  
SX0 X0+20 {C(X0) is replaced by C(X0)+20,  
least 18 bits only}

#### 10.6.4. Arithmetic and logical operations

Most of the arithmetic and logical operations are performed among the X registers. The general format, in COMPASS form is given below. These are all 15-bit instructions.

$$\left\{ \begin{array}{c} F \\ I \\ D \\ B \\ R \end{array} \right\} X_i \quad [-]X_j \left\{ \begin{array}{c} + \\ - \\ * \\ / \end{array} \right\} X_k$$

Here, F means “floating-point”, I means “integer”, D means “double precision”, R means rounded floating-point, and B means Boolean. The operator + means “add” for the arithmetic operations and logical OR for Boolean. Binary operator “-” means “subtract” for the arithmetics and exclusive-OR for Boolean. Unary operator “-” means “negate” for the arithmetics and logical complement (NOT) for Boolean. Operator “\*” means “multiply” for the arithmetics and logical AND for Boolean. Operator “/” means “divide” and is legal only for arithmetic operations.

The 60-bit numbers can be interpreted in different ways in this system, but the arithmetic units are essentially structured around a floating-point number representation. This representation contains a sign bit, an 11-bit biased exponent and a 48-bit integer coefficient. Double precision arithmetic is performed with two of these, such that one represents the most significant 48 bits of a value and the other the least significant 48 bits.

The real number representation includes special representations of  $+\infty$ ,  $-\infty$ , and “indefinite.” Infinity is obtained whenever a floating-point operation yields a result outside the maximum range of number representation. “Indefinite” is obtained through a mathematically ambiguous operation, e.g.,  $0/0$  or  $\infty - \infty$ . Once such special values are obtained, the arithmetic units can continue to operate on them through the usual arithmetic rules. For

example,  $\infty + n$  yields  $\infty$ , “indefinite” \*  $n$  yields “indefinite”, etc., where  $n$  is any finite value. Conditional tests on the infinite and indefinite number forms are also provided.

Rounded (“R”) or truncated (“F”) single-precision floating-point operations may be selected.

The integer instructions (e.g.,  $IX1\ X2 + X3$ ) interpret a 60-bit operand as a ones-complement integer. Unfortunately, the machine provides no means of testing overflow on an integer operation, and only integer addition and subtraction are provided. Integer multiplication and division require conversion to floating-point first. Accurate integer division requires several instructions, inasmuch as special attention must be paid to fixing the result correctly.

In ones-complement number representation, the negation of some positive binary number  $n$  is formed by complementing every bit. Thus  $-1$  is represented as

11111...110

As a consequence, there are two representations for zero,

00000...000

and

11111...111

Another consequence of this representation is that the least significant bit is 0 for an even positive number and for an odd negative number. Thus an even-odd test based only the least significant bit would fail.

The Boolean operations are the usual bit-by-bit AND, OR, exclusive-or (XOR), and complement (NOT).

Some examples of operation instructions:

|     |           |                                              |
|-----|-----------|----------------------------------------------|
| FX3 | $X3 * X5$ | {product of $C(X3)$ and $C(X5)$ to $C(X3)$ } |
| BX4 | $-X6$     | {complement of $C(X6)$ to $C(X4)$ }          |
| RX7 | $X1/X3$   | {rounded quotient $C(X1)/C(X3)$ to $C(X7)$ } |

### 10.6.5. Branches

A variety of branch operations are provided as follows. The unconditional branch JP is the only branch whose target can be indexed through a B register. Otherwise, the branch target is address K. The conditional branches ZR, NZ, etc., test the contents of  $X_j$ , then branch to location K if the test is satisfied.

JP  $B_i + K$  unconditional jump to  $(B_i + K)$

$\left\{ \begin{array}{l} \text{ZR} \\ \text{NZ} \\ \text{PL} \\ \text{NG} \\ \text{IR} \\ \text{OR} \\ \text{DF} \\ \text{ID} \end{array} \right\} X_i, K$  conditional jump to  $(K)$ ,  
 based on  $X_i$

$\left\{ \begin{array}{l} \text{EQ} \\ \text{NE} \\ \text{GE} \\ \text{LT} \end{array} \right\} B_i, B_j, K$  conditional jump to  $(K)$ ,  
 based on  $B_i$  relative to  $B_j$ .

RJ  $K$  return jump to  $(K)$

Here,

ZR: Register is zero.

NZ: Register is nonzero.

PL: Register is positive or zero.

NG: Register is negative and nonzero.

IR: Register is in range, i.e., not infinity.

OR: Register is out of range, i.e.,  $+\infty$  or  $-\infty$ .

DF: Register is defined.

ID: Register is indefinite.

EQ: Two registers are equal.

NE: Two registers are not equal.

GE: Left register  $\geq$  right register contents.

LT: Left register  $<$  right register.

Note that two branches are needed to test for “negative or zero” or “positive and nonzero.”

A pair of B registers may be compared with the EQ, NE, GE, and LT instructions. For example,

NE  $B_3, B_5, 150$

causes a branch to 150 if  $C(B_3) \neq C(B_5)$ . It turns out that this instruction form can be used as an unconditional branch and is faster than the JP, e.g.,



EQ B0,B0,K

unconditionally branches to K.

The return jump, RJ, is useful for Fortran-style, nonrecursive subroutine calls. The mechanism is shown in figure 10.22. Consider the instruction

RJ K

in location L. When executed, it writes the instruction

JP L+1

in location K, then branches to location K + 1. Location K should contain the first instruction of the subroutine. An exit from the subroutine is achieved by a branch to location K; this branch returns control to location L + 1, just following the RJ instruction.

We use the term “subroutine call” to describe this process, since RJ is totally unsuitable for recursive procedure calls in a block-structured language. A recursive subroutine call using RJ would destroy a previous JP stored in the dedicated memory location. The RJ instruction is not indexable, nor is it capable of storing a location in a register or stack. Its use also renders code nonreenterable; a section of code cannot be used by more than one process at a time.

#### 10.6.6. Other operations

The 6000 system contains several other instruction formats not discussed here. These include shift, normalization, packing, unpacking, ones count, and mask formation instructions. These are of value for number conversion and bit-field logical operations.

#### 10.6.7. Procedure Parameters in a Fortran Implementation

Since recursive subroutine calls are forbidden in Fortran, it is sufficient to allocate registers or memory at compile time for the subroutine local variables and formal parameters. Parameters are always passed by reference, so a list of 18-bit addresses would be sufficient as formal parameters.

The Fortran convention on the 6000 series is to use registers B1 to B6 for the addresses of the first six parameters. The A and X registers are assumed to contain garbage upon entering a subroutine, may be used freely within the subroutine, and need not be restored upon an exit.

If a subroutine contains more than six parameters, the addresses of the extra parameters are written to memory locations just preceding the target of the RJ instruction that calls the subroutine. The arrangement is shown in figure 10.23.

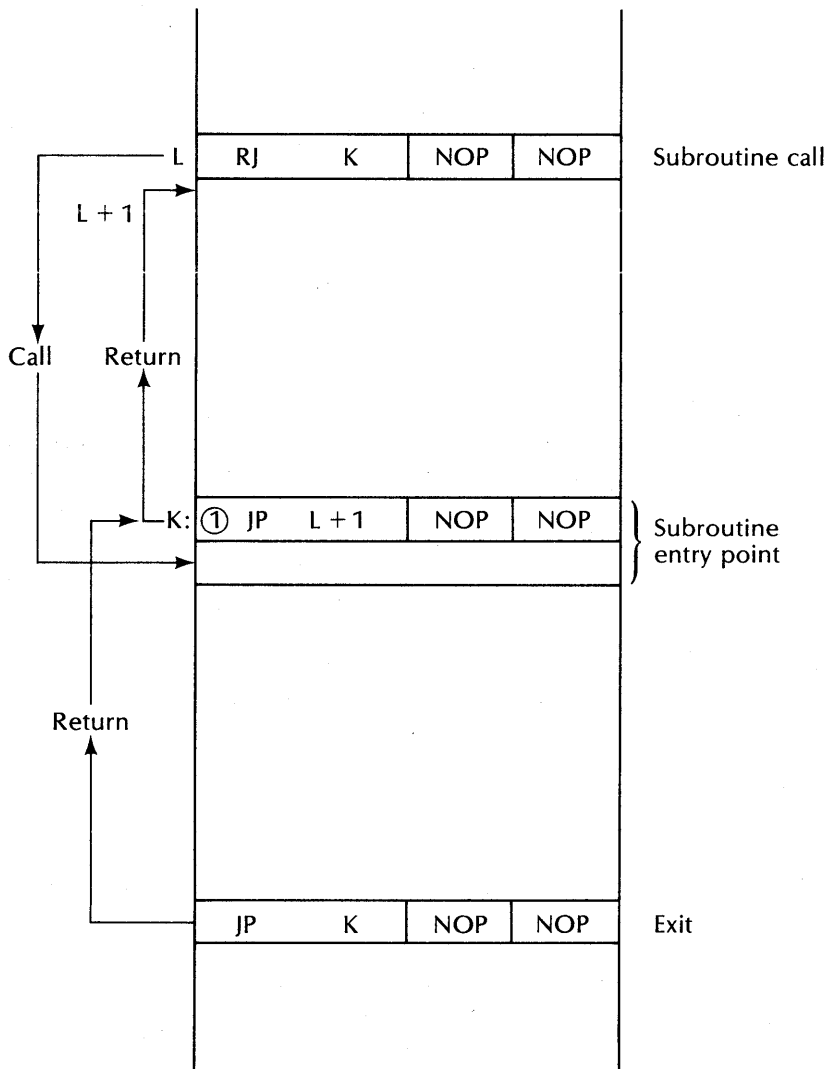


Figure 10.22. Return jump RJ instruction operation.

A typed subroutine returns its value in one of the X registers. No address of the return is necessary.

10.6.8. Arithmetic Expressions

Arithmetic expression evaluation requires a small stack of temporary locations; these could be allocated from the X-register set and memory for the purpose. We can then conceive of a general purpose register allocation system

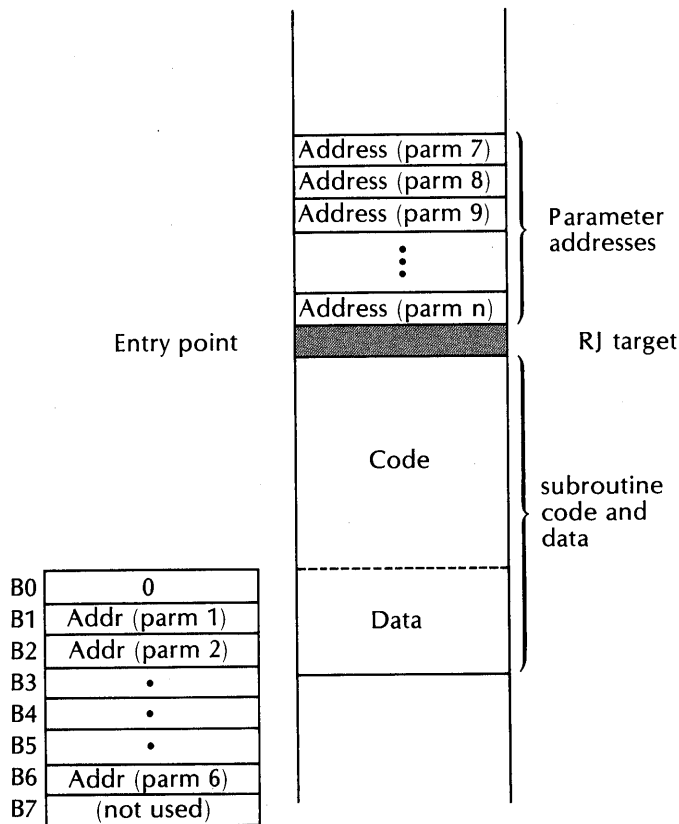


Figure 10.23. Fortran subroutine parameter passing conventions (simplified).

that manages all seven of the X registers. The allocator is of course complicated by the three different roles played by the sets {X0}, {X1...X5}, {X6, X7}. In general, the allocator must attempt to keep variables and temporaries in X-registers as much as possible in the interest of efficiency, yet deal with register allocation by storing some results in order to make room for others. Of course, the instruction form BXi Xj facilitates its housekeeping; an X register can always be stored with two 15-bit instructions provided that X6 or X7 (and A6 or A7) is free.

Chapter 11 discusses some register allocation systems that might be applied to this architecture.

### 10.6.9. Saving and Restoring Registers

The 6000 system has an interesting deficiency—it is very difficult for a user program to save (and later restore) the contents of all the registers. In saving

the registers, the essential problem is how A6 (or A7) can first be saved without losing vital information. The problem can be solved, but requires a large number of instructions. The key to its solution is a series of 18 operations. Each operation consists of a conditional branch based on the sign of X1 (say), that conditionally causes an RJ and a left shift of X1. The RJ's are used to leave a trail of "marks" in memory that indicate whether the sign was 0 or 1. After 18 such operations, enough room is left in X1 for A6, and from there on, the rest is easy.

The difficulty of saving and restoring all the registers implies that a complete stack emulation is impractical; at least one register will have to "float." In practice, most of the 6000 software is coded with the convention that the caller of a subroutine is responsible for saving important register contents. A subroutine call will destroy any or all register values in general.

Of course, the machine status, including all register contents, is saved through the *exchange jump* (EXJ) instruction. However, this instruction can only be executed by a peripheral processor, and is only invoked when control of the CP is transferred to another process. EXJ is not available to a user.

#### 10.6.10. Relocatable Linking and Partial Compilation

In Fortran, each subroutine in a set comprising a program is separately compilable in the 6000 system. In practice, small programs can be compiled so rapidly and cheaply that partial compilation is not often used. However, the capability exists and deserves some discussion.

The Fortran compiler arranges that every subroutine reside in a contiguous block of words (figure 10.23). The block begins with parameter space (if more than six parameters are needed), then a word for the RJ target, and then the subroutine code and data space. (Actually, a parameter count and the subroutine name are also present, for the sake of the loader.) Since all memory references are full 18-bit addresses, all the data can be collected at the end of the block, so that the instructions need not be interrupted by data.

The compiler prepares each subroutine block in the general form shown in figure 10.23, and generates code under the assumption that the block begins in location 0. It then prepares tables for the relocatable loader that specifies

1. The name of this subroutine.
2. The entry point location (target word for a calling RJ).
3. The location of every memory reference instruction (containing a K field) in the block. The location must include a position code specifying the position of the instruction in the word.
4. The name associated with every subroutine call and a pointer to the call. The RJ instructions associated with one subroutine are linked together. The system is similar to that in the HP-3000 system (figure 10.15).

The loader must adjust each of the K-reference instructions by the offset of the subroutine block within the program block. Also, every subroutine call (an RJ somewhere) must be fixed to point to the target subroutine.

Since partial addresses and indirect words are not needed, these loader operations are quite straight-forward and efficient. Of course, loading need only be done once after a compilation. The loaded program block can be saved and executed with no setup time at all.

### 10.6.11. A Pascal Implementation

Wirth [1971] describes an implementation of Pascal on the 6000 system. He found that several of the 6000 features can be exploited to yield an efficient implementation of this block-structured language supporting recursive and nested procedures.

Wirth's Pascal compiler generates absolute code (it uses the relocatable loader in a trivial way). The RJ instruction is never used, because it fails to support recursive procedure calls. The allocated memory program area is divided among the program, the stack, and the heap. Of course, these three comprise one program area to the operating system, with lower bound RA and extent FL.

The registers X1 to X5 are used as a stack to hold intermediate results during the evaluation of expressions. Wirth found that X1 was used 75% of the time, but X5 only 0.02% of the time, out of the 7927 compiled instructions that assign values to one of these registers. This means that very few expressions require more than 5 intermediate results. The Pascal compiler in fact declares a user error if more than 5 intermediate results are needed in some expression. Removal of this restriction through assignment of memory temporaries could be done, but would add to the compiler's complexity and yield a marginal benefit. Registers X6 and X7 are used to store results in operand fields in the stack or heap.

Registers B1 to B4 are used for the display (see chapter 9). These provide four nesting levels, of which only three are required for the Pascal compiler. Since new local variables can only be introduced by declaring a new procedure in Pascal, the compiler has at most three static procedure nesting levels.

The use of a B register as a display is especially attractive on the 6000. The set instruction form

$$SA_i \quad B_j + K$$

is useful for fetching or storing local variables, and the form

$$SA_i \quad B_j - K$$

is useful for fetching or storing formal parameter values. Parameters passed by reference, or pointers, can be accessed through an  $SA_i B_j - K$  instruction, followed by an  $SA_i X_i$  instruction.

Indexing through the display is achieved through the following sequence of instructions:

```
{evaluate index in register  $X_i$ }
SB5  $X_i \pm K$  { $K$  is the parameter
               offset from display}
SAi  $X_i + B_k$  { $B_k$  is the display}
```

The last instruction replaces the index in  $X_i$  by the data value.

A procedure call is implemented essentially as discussed in chapter 9, except that Pascal supports neither call-by-name, nor branch label parameters. The necessary words to be written to the stack by the PE and BE operations are then only the dynamic link (procedure return address) and the static link. A 60-bit word can carry up to three address labels, so that a one-word stack marker is feasible.

Since the compiler generates absolute code, each of the procedure's addresses is known at compile time. The dynamic link can be written in two instructions as follows, where  $K$  is the return address and  $K' + B_j$  is the stack location to which the address is to be written:

```
SX6  $B_0 + K$  {writes  $K$  to  $X_6$ }
SA6  $B_j + K'$  {written to the stack}
```

The *powerset* type in Pascal provides a set membership facility for a finite set. The membership of some set element in a set  $S$  can be represented by a 1 or 0 bit in position  $c_i$  in a word assigned to that element. Then the machine's bit-parallel logical instructions ("and" for intersection, "or" for union, "not" for set inverse) can be used for set operations. A test for set membership is equivalent to a word shift followed by a sign-bit test. Wirth's compiler restricts the number of set elements to 60, but this is again a restriction that could be lifted through some additional work.

### 10.6.12. Summary

The major defects of the 6000 system for Pascal (or any other Algol-like language implementation, for that matter) are

1. The absence of a jump instruction that deposits the current processor status in a general register or in a register designated stack location.
2. The absence of logical shift operations. (All the shifts are arithmetic. They propagate the sign bit on a right shift, or left-circular on a left

shift.) A true logical shift would be desirable for the set membership test.

3. The use of one's complement arithmetic with two representations of zero. (Zero is either all zeroes or all ones).
4. The failure to indicate overflow in integer arithmetic and the different arithmetic magnitudes of the various integer instructions.
5. The absence of character-handling operations, needed in languages such as Snobol, Cobol, and RPG. Character-handling operations are supplied in the CDC Cybernet 7600 systems.

Apart from these defects, the 6000 system is among the most powerful and fast computer systems in existence and is unparalleled in the execution of large, floating-point, scientific programs. The simplicity and generality of its instruction set makes possible relatively simple compilers. However, optimization of the use of its register sets is a difficult problem, especially in this system, considering the memory access properties of registers A1 to A7.

### Exercises

1. Describe in greater detail an implementation of PE, BE, EB and a procedure call, using the Pascal conventions and the 6000 instruction set. Assume that a packed stack marker is not necessary.
2. Develop systems for the allocation and access of local variables in a Pascal system, both static and dynamic.
3. Define the Fortran subroutine structure in more detail, and give a loader algorithm in Pascal.
4. Write a Compass (6000 assembler) subroutine that performs integer division, and yields the quotient and the remainder, for positive integers. (Access to a Compass manual and the 6400 reference manual would be helpful.)
5. Sketch an algorithm that lifts the restriction that an expression evaluation may invoke at most five temporary values.
6. Sketch an algorithm that lifts the restriction that at most five nesting levels are permitted. *Note:* Assume that B6 and B7 are needed as general purpose registers.
7. Examine the problem of saving and restoring all the registers. (Access to the complete instruction set would be helpful in this exercise. However, you may assume that the machine contains left and right arithmetic and circular shifts on the X registers.)
8. Given that only the field length FL may be altered during execution,

discuss the problem of dynamic allocation of memory for a Pascal compiler that requires a stack and a heap on this system.

## 10.7. The IBM System/360

The IBM System/360 is actually a set of compatible computer systems, covering a wide range of price and performance. The internal performance range between the largest and the smallest 360 system is approximately 50:1 for scientific computation and approximately 15:1 for commercial processing.

System compatibility is achieved through a common instruction set and virtual memory in all models. In the smaller models, some of the instructions are performed through software, and large programs generate considerable disk access.

Each system is based on a single CPU that has access to main memory and I/O channels. We will not be concerned with I/O access, as most of that is handled by the operating system, and may be set up with standard macros.

The CPU is time-shared between operating system and user operations, with operating system and I/O channel operations receiving higher priority in general.

Multiple CPU's can be interconnected and can access the same memory. Multiple CPU's can be used in a multiprogrammer system by assigning each one to a different task through a scheduler. The tasks may be part of one large program or separate user programs, but must function asynchronously.

A CPU contains 16 general purpose registers of 32 bits each, and four floating-point registers of 64 bits each. These are interconnected through system control to the main storage, which can consist of as much as 16.5 million bytes.

The 16 general registers are designated 0, 1, ... , 15, and the four floating-point registers are designated 0, 2, 4, 6. Although the same codes are used for the two sets of registers, different instructions are provided for loading and storing from the two different register sets and for operations on the register contents.

Long (64-bit) and short (32-bit) floating-point arithmetic is provided, with packing and unpacking operations. A full set of arithmetic operations on both forms is provided.

Integers may be fullword (32 bits) or halfword (16 bits). All fixed-point arithmetic is 32-bit two's complement, however. All 16-bit arithmetic is performed with 32-bit registers by setting carry and overflow indicators relative to the least significant 16 bits.

Instructions are also provided to process decimal data, in the form of packed 4-bit binary-coded-decimal characters, or zoned 8-bit characters. The



operands for decimal data reside in memory; the instructions carry addresses of the left-most operand bytes.

Data in memory is subject to alignment restrictions, as shown in figure 10.24. Main memory is byte organized; every address is a byte address. A halfword must be aligned on an even byte boundary, a fullword on an even halfword boundary, and a double word on an even fullword boundary. These restrictions apply to all the memory reference instructions. The alignment restrictions can only be broken through byte move instructions that can move a non-byte entity between an even and an odd location in memory.

A special 64-bit status register PSW carries several fields of machine status information, including the next instruction address, overflow, and carry indicators, and a *condition code*. The condition code is a 2-bit field in PSW that is set by various arithmetic operations and sensed by a set of conditional branch instructions. The code value 0, 1, 2, or 3 depends in a specific manner upon the instruction that affects it. Its interpretation through a conditional branch instruction is by a 4-bit field in the branch instruction; the bits correspond to codes 0, 1, 2, and 3, and the branch is taken if the bit corresponding to the current condition code is set. For example, if the condition code is 2, then any branch instruction bit pattern of the form XX1X causes a branch.

Program interrupts are possible as a result of certain operations, such as fixed- or floating-point overflow, or an address bounds violation. When a program interrupt occurs, the current instruction execution is suspended and an operating system service routine is called. Overflow interrupts may be

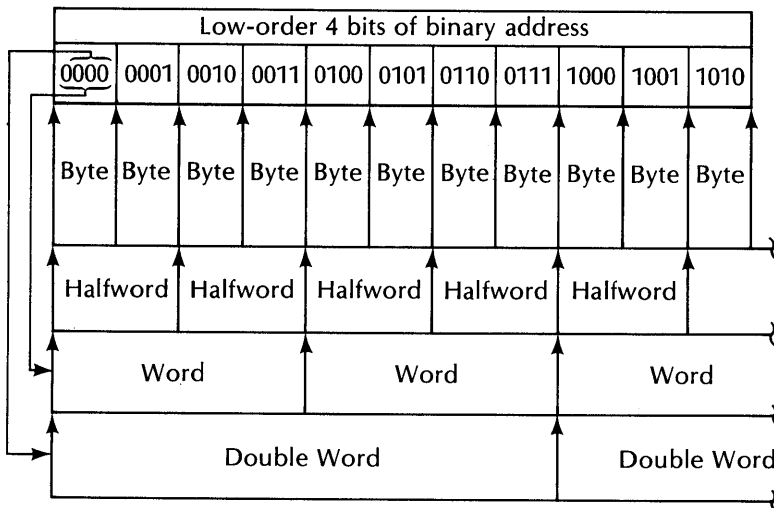


Figure 10.24. Data fields and alignment in memory for IBM 360.

brought under user control, if desired, through mechanisms that we shall not discuss.

### 10.7.1. Instructions

The five 360 instruction formats are illustrated in figure 10.25. Instructions are 1, 2, or 3 halfwords in length, depending on the op code, and must be aligned on halfword boundaries in memory. Memory is shared between instructions and data in any desired manner.

An operation code occupies the first 8 bits of every instruction. The remainder of the instruction designates one or more registers (R, B, X), and possibly one or two address displacements (D). The SI format carries an 8-bit immediate operand (I2). The SS format carries one or two length operands, L or (L1, L2). The SS format is used for memory operands, e.g., decimal arithmetic, byte moves, and byte string comparisons.

A “0” as an index or base register designation means that no register is used in that part of an address calculation. However, register 0 can carry a data value and be accessed by instructions that refer to the register’s contents as data rather than as an address component.

Scalar nondecimal operations are between two registers, or between a register and a memory location. The RR instruction format (figure 10.25) always specifies a register-to-register binary operation. Thus

OP R1,R2

for a binary operation OP is equivalent to

$$C(R1) \leftarrow C(R1) \text{ OP } C(R2)$$

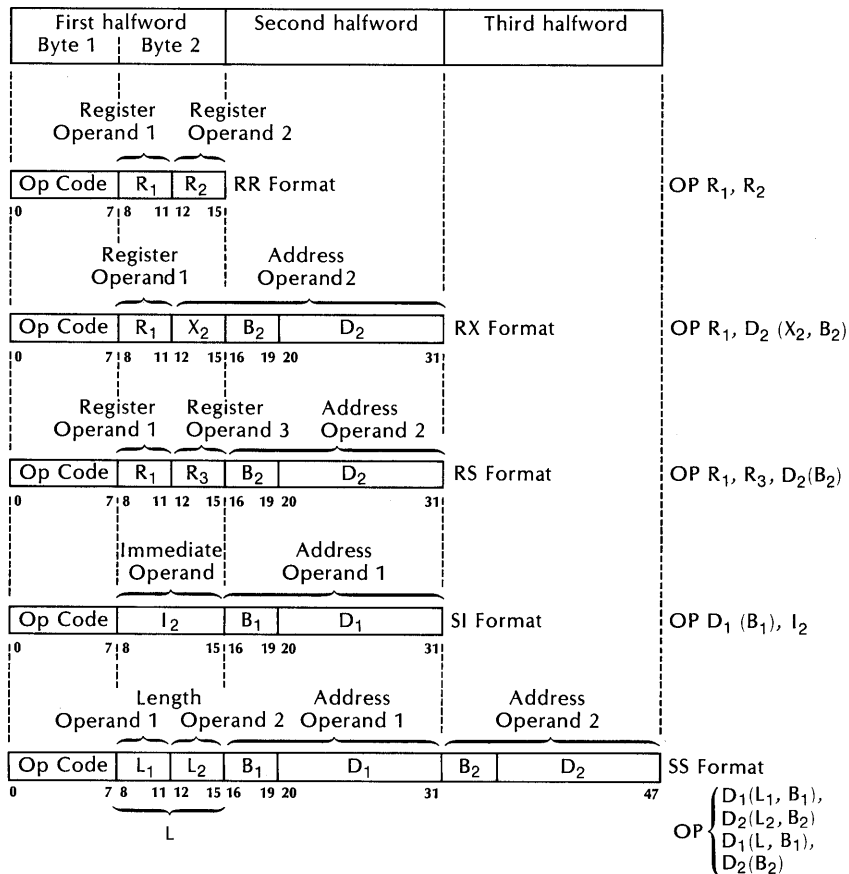
The RX format specifies a register-to-memory operation in general; it is written symbolically as

OP R1,D2(X2,B2)

The first operand is C(R1). The second operand is

$$C(D2 + C(X2) + C(B2))$$

Here,  $C(X2) = 0$  if  $X2 = 0$ , and  $C(B2) = 0$  if  $B2 = 0$ . That is, the contents of X2 and B2 are added to the displacement D2 to form an absolute memory address of the second operand. Although the functions of X2 and B2 in this instruction are interchangeable, X2 is usually considered the *index* register, in the sense that it might be used to access an array element, and B2 is



**Figure 10.25.** The five basic instruction formats for IBM 360.

considered a *base* register. The base register is needed to specify the loading origin of the program and cannot be used as a second index register.

The displacement D2 provides relative addressing up to 4095 bytes (the displacement field is 12 bits) beyond the base and index register address. Each of the three address components are treated as positive numbers. However, address arithmetic is performed without regard to overflow, so that the effect of negative indexing is achieved by regarding the index register as a positive or negative two's-complement 24-bit number. For example, if  $C(B2) = 43000$ ,  $D2 = 1600$ , and  $C(X2)$  carries the (hexadecimal) value  $FFFFFFFC$  the effective address is  $43000 + 1600 - 4$ , since  $C(X2)$  represents  $-4$  in two's complement form.

The effective address must lie within a memory partition assigned to a user,

or else an interrupt will occur. This provision protects one task in memory against an unauthorized access of its memory by another task. Only certain operating system procedures have unrestricted access to all the system's memory. Protection is achieved by dividing main storage into pages of 2048 bytes each, and by associating a 5-bit key with each page. A user's right of access to storage is identified by a 4-bit protection key. Access to some storage location is only granted when the user's protection key matches the storage page key. A violation causes the instruction to be suppressed or terminated and execution to be altered by an interrupt.

When a task is initiated or resumed, the memory protection keys are all set to delimit a section of memory for the task; usually, the task is permitted to access only one contiguous partition of memory for reading or writing of data. The memory partition assigned to the task is determined when the task is initialized and is frozen in location until it is completed. However, the partition may be rolled out and later rolled in asynchronously by the operating system for the sake of overall system scheduling needs.

### 10.7.2. An instruction sample

A few of the 360 instructions are described below. A complete instruction set may be found in any of several references and in the IBM System/360 Principles of Operation manual.

#### Register Load

```
LR  R1, R2          Load
L   R1,D2(X2,B2)
```

The second operand is placed in the first operand location. The second operand and the condition code are unchanged.

```
LH  R1,D2(X2,B2)    Load Halfword
```

The halfword second operand is placed in the first operand location. The second operand is expanded to a full-word by propagating the sign-bit value through the upper 16-bit field.

The LH instruction copies a halfword (16 bits) in memory specified by D2, X2, B2 to register R1. The halfword operand is expanded to form a full 32-bit two's-complement arithmetic operand by propagating the sign bit into the 16 high-order bit positions. The condition code is unaffected, but an addressing interrupt may occur.

**LTR R1,R2      Load and Test**

The second operand is placed in the first operand location, and the sign and magnitude of the second operand determine the condition code. The second operand is not changed. The resulting condition code is: If result is zero, then 0. If result is  $<0$  then 1. If result is  $>0$  then 2.

The LTR instruction is identical to the LR instruction, except that the condition code is set according to the sign and magnitude of the operand, as indicated in the figure.

**Other LOAD instructions**

Twenty-two LOAD instructions are provided in all, for all the fixed- and floating-point data types provided in the system. Some provide negation or complementation during the copy. Thus the LPSW, load program status word) instruction loads the system's 32-bit status word PSW into a selected register. The LA instruction loads the 32-bit address of the operand into the designated register. The LM instruction loads a group of the general registers from a contiguous group of memory locations. The LM instruction and its complement STM facilitate saving and restoring the general registers. One byte can be loaded or stored through the IC or STC instructions respectively. IC is not a true load instruction, as it only changes the least significant byte in the register; the most significant bytes are unaffected.

No LOAD instructions are provided for decimal operands, inasmuch as these have variable lengths and are operated upon directly in memory by SS instructions. Of course, parts of a decimal operand can be loaded or stored through the byte and word load/store instructions.

**Fixed- and Floating-point Arithmetic**

**AR R1,R2      Add**  
**A    R1,D2(X2,B2)**

The second operand is added to the first operand, and the sum is placed in the first operand location. Addition is 32-bit 2's complement. The condition code is set: if sum is zero then  $CC=0$ ; if sum is  $<0$  then  $CC=1$ ; if sum is  $>0$  then  $CC=2$ ; if overflow then  $CC=3$ .

The AR and A instructions are typical of the arithmetic instructions: subtraction, multiplication and division of all the fixed and floating-point data have similar instruction forms.

**Comparisons**

**CR R1,R2      Compare**  
**C    R1,D2(X2,B2)**

The first operand is compared with the second operand, and the result determines the setting of the condition code. Operands are regarded as 32-bit signed 2's complement integers. If the operands are equal then  $CC=0$ ; if first operand is low then  $CC=1$ ; if first operand is high then  $CC=2$ .

The compare instructions CR and C specify two operands, but only set the condition code. The operands are unaffected. An addressing interruption is possible on C.

### Branches

```
BCR  M1,R2      Branch on Condition
BC   M1,D2(X2,B2)
```

The updated instruction address is replaced by the branch address if the state of the condition code is as specified by M1; otherwise, normal instruction sequencing proceeds. The branch address is specified by the second operand R2, or  $D2 + (X2) + (B2)$ . The M1 field is used as a four-bit mask; each bit corresponds, from left to right, to one of the four possible condition codes. When all four M1 bits are set, the branch becomes unconditional; when all four are clear, the instruction is a no-operation.

For example, if the current condition code is 2 and the mask carries the binary value 0010, then the condition is satisfied and the branch occurs. The branch address is either in register R2, for BCR, or is specified by the usual set (D2, X2, B2), for BC.

A program interruption will occur on the next instruction, if its location is outside the range of accessible memory.

### Subroutine Call

```
BALR  R1,R2      Branch and Link
BAL   R1,D2(X2,B2)
```

The rightmost 32 bits of the PSW (program status word), including the updated instruction address, are stored as link information in the general register specified by R1. Subsequently, the instruction address is replaced by the branch address. If the R2 field contains zero, the link information is stored without branching.

These are useful for subroutine calls and for setting a program address in a base register. The right-most 32 bits of the PSW will carry the byte address of the next instruction (following BALR or BAL) when it is loaded, hence register R1 will be loaded with a "return address". The branch target is specified by the contents of register R2, for BALR, or the set (D2, X2, R2), for BAL. As usual, an address interrupt will occur on the next instruction if it lies outside accessible memory.

### Other Instructions

The few instructions defined above should provide some flavor of the System/360 architecture. There are 142 instructions altogether. These cover arithmetic, comparisons and conversions of all the data types. There are in addition logical operations (OR, AND, XOR, NOT, shifts) on the fixed-point types, several kinds of moves, some I/O, and some special edit instructions to facilitate business report writing.

Since our concern is primarily with register and addressing conventions, we shall not define more instructions.

### 10.7.3. Procedures and program relocation

Procedures may be independently compiled and later linked together. A compiler prepares a file called an *object module* for each procedure. A set of object modules are then linked by a linking loader to form a *load module*. A load module can then be assigned and written to a region of memory just before execution by the absolute loader. A load module may require a few adjustments, as we shall see, or not, depending on the source program requirements.

Two kinds of relocation will occur in general, and they are handled by the *linking loader* and the *absolute loader*.

The *linking loader* brings together several object modules into a single load module. In general, there will be many subroutine call instructions and memory addresses that must be adjusted to conform to the locations assigned to the subroutines comprising the module. The linking loader cannot necessarily make all the adjustments needed for relocation, and therefore must prepare a special relocation table for the absolute loader.

The *absolute loader* accepts a load module and positions it somewhere in memory. It must adjust certain address constants to reflect the memory position to which the program has been assigned; these constants are listed in a table prepared for it by the linking loader.

Relocation is achieved in two ways—through a *base register* convention, and through adjustments to *address constants*. A base register is a designated register (usually number 13) that contains the absolute loading address of the program in memory. Every memory reference instruction must contain a base register component, so that every reference will be correctly made during execution. Failure to specify a base register on some instruction or failure to establish or maintain the relocation in a base register will cause a bounds violation or other program failure during execution.

An *address constant* is some memory fullword that contains the absolute program loading location as a component. Such addresses must be adjusted by the absolute loader just prior to execution. As we shall see, address constants are an important part of 360 run-time structures.

#### 10.7.4. Save Areas

Every program module that will pass control to some operating system control program and back again must provide a *save area*. A save area is comparable to the stack marker in the AOC machine and is a contiguous block of 18 fullwords. It “belongs” to the calling program, not to the callee, yet the callee uses it to save registers. Figure 10.26 shows a save area belonging to a procedure Y, called by procedure X. When Y calls a procedure Z, Z fills Y’s save area.

The save area contains 15 fullwords for the general registers 0 to 12, 14 to 15, a fullword for the address of the previous save area, a fullword for the address of the next save area, and a fullword used by PL/I.

The address of a caller’s save area is passed in register 13. The callee then bears the responsibility for filling the save area, and restoring the register contents upon an exit. The register set 0 to 12, 14 to 15 can be saved (restored) by a single STM (LM) instruction.

Recursive or reenterable programs require allocation of a new save area upon each call and its deallocation upon return. Otherwise the save area may be a fixed component of a program data space.

The forward and backward chains through save areas facilitates debugging and abort analysis; the chain provides useful information regarding the procedure call nesting, the entry and exit parameters, and return addresses.

Register 14 is assumed to carry the return address upon a procedure call. Register 15 is assumed to carry the entry point address of the called

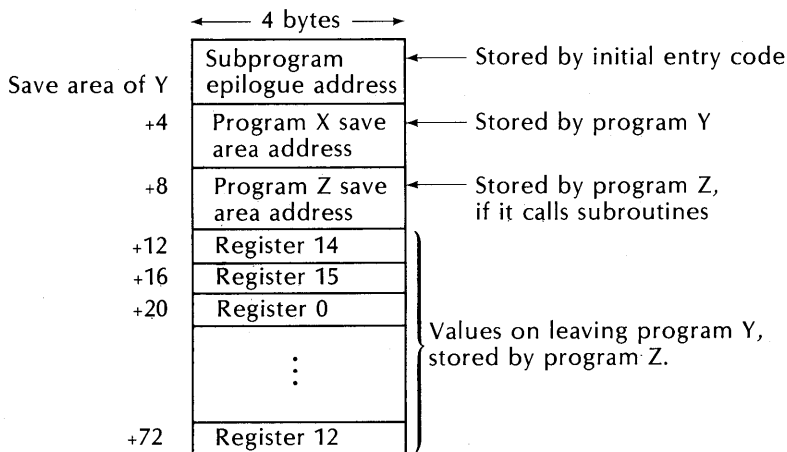


Figure 10.26. A save area.



procedure. Registers 0 and 1 are generally used to pass the address of a parameter list to the called program. These and register 13 are called *linkage registers*. Although the machine architecture requires no special register use conventions, the operating system control programs, debug and dump systems, and the system macros do depend on these conventions.

Upon a return from a procedure, the registers must in general be restored to their state upon entry. However, register 15 can be used to return a *return code*, useful in indicating any unusual conditions or errors found during execution of the procedure. A return code of zero is interpreted by the system as “no errors—nothing unusual”. Any other code can be used, but of course should be documented. Registers 0 and 1 need not be restored; recall that they were used to pass parameter information to the procedure. Register 14 contained the return address upon entering the procedure, so the return is effected by a BR 14, executed after restoring register 14.

### 10.7.5. Object Modules

A compiler prepares an object module for each procedure in general. An object module consists of three tables—ESD (External Symbol Dictionary), TXT (Text), and RLD (Relocation and Linkage Directory).

The TXT table is a list of fullwords to be initially written to memory. Instructions, constants, initialized, and uninitialized data are specified through a TXT table. Some of these fullwords must later be modified by the linking loader and by the absolute loader. An initial loading address of 0 is assumed by the compiler.

The ESD table contains every symbol defined in this segment that must be referenced externally, and every symbol defined elsewhere that is referenced in this module. Three kinds of ESD entry are needed as follows:

| Symbol | Type | Location | Meaning            |
|--------|------|----------|--------------------|
| RESULT | SD   | 56       | Module definition  |
| SUM    | ER   | 864      | External reference |
| COMP   | LD   | 270      | Local definition   |

RESULT is a name given to the module being constructed, and it is useful as an entry point location definition (in this case, the first executable location is 56) and as a name for use in a symbolic loader map. The 360 loader will support multiple entry points into a given module, so there could be several of these.

The address of SUM is needed in location 864, however SUM is externally defined. The loader must therefore determine the address of SUM and fill location 864 with it. The ER type may also be used for calls of external procedures; a procedure is called on the 360 by loading register 14 with its

address (from memory, where it was set up by an ER entry in the ESD table), then executing BR 14 (actually BCR 0,14).

COMP is defined locally, but its address is needed externally.

The RLD table is used exclusively for address constants appearing in the program segment. Recall that an address constant is some memory word whose value depends on the location of this program or some external program. For example,

DC    A(PG1 - PBXX + SAM - 14 + FRED + 17)

might appear in an assembly language program. The mnemonic DC means *define constant*, and A( . . . ) means “address of ( . . . )”. Here, the constants 14 and 17 would be combined by the assembler. This constant would then appear as a “3” in some location, say 374. Then four RLD entries, one each for PG1, PBXX, SAM and FRED, would be generated, all referring to location 374.

The RLD table contains the following information:

- The location and length of each address constant that must be modified.
- The external symbol by which the address constant must be modified.
- The modification (addition or subtraction).

Several RLD entries may refer to the same address constant. Each of them causes some modification to the constant—an addition or subtraction of an externally defined address. Thus the above DC would result in a DC 3 in the TXT table, and four RLD table entries for PG1, PBXX, SAM, and FRED.

An address constant as seen by the absolute loader will then be one of the following:

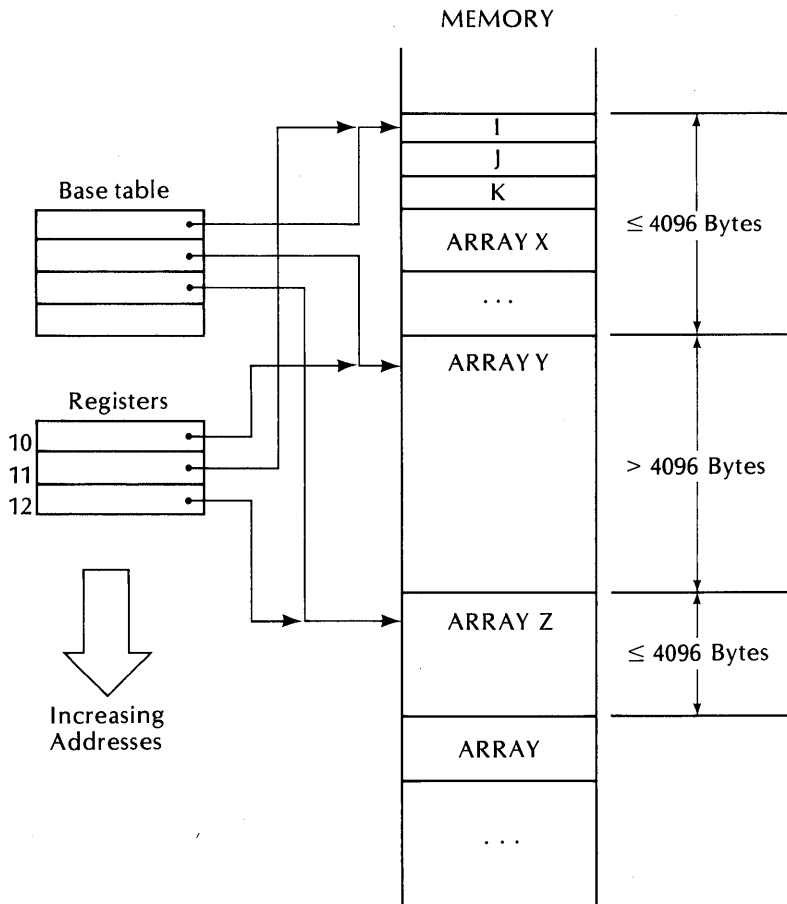
1. *Absolute*. Will occur if addresses in an address expression appear in pairs, one added, the other subtracted. The absolute loading location of the load module will therefore not affect the address constant.
2. *Simple relocatable*. Will occur if all but one of the address expressions appear in additive-subtractive pairs, and the one left over is an additive address. The absolute loading address of the load module must be added to the address constant by the absolute loader.
3. *Compound relocatable*. Will occur if the absolute loading address of the load module must be added to the address constant twice or more, or subtracted once or more times.

Clearly, the absolute loader need not be informed of case 1 at all—the linking loader was able to resolve the address constant completely. Case 2 is very common, and requires one RLD entry. Case 3 is likely to occur only in unusual assembly language programs; it requires more than one RLD entry.

### 10.7.6. Addressing

The 360 memory reference instruction carries a base register, an index register and a displacement (see figure 10.25). Now the displacement field is relatively small, 0 to 4095 bytes. If a procedure carries more than 4096 bytes of data, then parts of that data cannot be accessed through a single fixed base register alone.

A solution to data addressing, when more than 4096 bytes of data must be accessed, is illustrated in figure 10.27. Here, data resides in some contiguous block of memory. Simple variables are grouped together, followed by arrays and other indexed structures. Another section of memory is dedicated to a *base table*, which contains address constants that point into various regions of



**Figure 10.27.** Base table and use of base registers to access data.

data memory. For example, the first base table entry might be a pointer to the first data cell. This base address, when placed in a register, can then serve to access any data element or array whose first location is less than 4096 bytes from the base address. Thus variables I, J, K, and all of array X (X can be any size) are accessible through the first base table address.

Subsequent base table entries are addresses of data memory farther than 4095 bytes from the previous one. Thus, if array Y is the first data area located more than 4095 bytes from variable I, then a base table element must be allocated to point to it. Here, Y can again be of any size, since a 32-bit register is available for indexing from the base table address.

For example, if registers 10, 11, and 12 are available as data base registers, then at any one time, each of these will in general point somewhere into data memory. If some data memory element is needed that cannot be reached from one of the registers 10 to 12, then one of these must first be replaced by a base table element.

The optimum assignment of registers to base addresses is clearly an interesting and difficult problem. For a very small program, some or all of the variable values themselves may be carried in registers, obviating many memory references. A larger (but not too much larger) program would permit one data base register through which all data accesses are possible. A still larger program requires a base table and dynamic setting of the base registers from the base table.

### 10.7.7. Object Module Design

It is best to establish an overall design of the object module and conventions for the use of the registers before anything else in a compiler is designed. We shall outline the design of the Fortran IV (G) compiler released by IBM in the late 1960's. We shall not describe the parsing and other algorithms in detail, but will look at some of the conventions of data allocation and code generation used in this system.

The arrangement of the program module, as generated by an absolute loader for one Fortran subroutine, is given below.

- Heading (entry point)
- Save area
- Base table
- Branch table
- EQUIVALENCE variables
- Scalar variables
- Arrays
- Subprogram argument lists
- Subprogram addresses
- NAMELIST tables

Literal constants (except those used in  
DATA and PAUSE statements)  
FORMAT statements  
Temporary storage and constants  
Program text

To a compiler, the program module is a TXT file. It must be modified by the linking and absolute loaders prior to execution.

Control passes into the first word of the program module. Some preamble instructions are in the *heading*. Among other things, the heading code sets general register 13 and saves all the registers.

The save area and the base table have been described above. The base table will contain absolute addresses during execution, and must therefore be adjusted by both loaders prior to execution.

The *branch table* contains absolute branch addresses, one for each branch target. Each branch target is assigned a fullword in the branch table, and a branch is then executed by loading a register R from this table, then executing BCR 0,R. The use of a branch table facilitates compilation in one pass without fixup; the branch instructions may be emitted in final form as they are encountered, and the branch targets may be entered in the branch table in suitable form as they are encountered.

The next three tables (equivalence variables, scalar variables, and arrays) carry the subroutine data. The equivalence variables are not dimensioned. An average Fortran subroutine will not have too many simple variables and nondimensioned equivalenced variables, so that these two tables can usually be accessed through one base register. The array table may require more base registers, as suggested by figure 10.27.

The next table (subprogram argument lists) carries the addresses of the arguments for all subprograms called by this subroutine. If N different subroutines are called, then there are N lists in this table, one list for each subroutine. Upon calling a subroutine, its address list is first prepared, then the address of its address list is passed in general register 1. The called subroutine can then access its parameters indirectly through register 1.

The next table (subprogram addresses) contains one entry for each subroutine called by this module; the entry holds the subroutine address, as supplied by the linking loader.

The *namelist* table is used for the NAMELIST feature provided in this Fortran version. NAMELIST is a Fortran feature for the symbolic access of program input/output file data.

Literal constants are held in the next table. These are sharable; if a "20" appears as an arithmetic constant in several places in the subroutine, it only appears once in this table.

The FORMAT statements appearing in the subroutine are coded in a

condensed form and placed in the next table. FORMAT is used by a run-time program to control the form of input and output, and its conversion to/from internal form. The run-time program interprets a list of characters in the FORMAT table.

The last data table consists of memory cells allocated for intermediate results and other constants. Registers are used as much as possible for intermediate results, for obvious efficiency reasons. However, only four registers are available as accumulators for intermediate values, because of other register uses. This data area provides intermediate result storage for any surplus beyond four values.

The program instructions occupies the last program module area. The first entries are occupied by statement functions, if any, with a branch around these to the first program instruction.

#### 10.7.8. Register Allocation

The four floating-point registers are used as needed as floating-point accumulators. The sixteen general registers are allocated as follows:

Registers 0, 2, 3 are used as accumulators. Recall that register 0 cannot be used as an address register in an instruction. These registers are saved in a subroutine call and can therefore hold intermediate results preserved in a call.

Register 1 is used as an accumulator within a subroutine, but also to pass the address of the argument list when calling a subroutine.

Registers 4 to 7 contain index values as needed for reference to array variables, where the subscripts are linear functions of DO variables and the array does not have variable dimensions. The Fortran IV (G) compiler provides some optimizations for array indexing in DO statements; an array index that is incremented by a constant amount on each iteration of the DO loop is carried in one of these registers and incremented by the amount on each iteration with one instruction.

Registers 8 and 9 contain index values as required for references to array variables, where the subscripts are of the form  $x \pm c$ , where  $x$  is a non DO-controlled variable and  $c$  is a constant.

Register 9 contains index values as needed for nonlinear subscripts, not covered by the one of the above cases. The motive for distinguishing these three cases appears to be to subdivide the array indexing-DO loop cases for optimization.

Registers 10 to 12 contain base addresses loaded from the base table (see figure 10.27). If the subroutine has more than three base table entries, these registers must be dynamically changed, otherwise they may be initially loaded from the base table and never changed.

Register 13 contains the address of the object module save area, following the system conventions outlined previously. It is also useful as a base table